Language models

Large Language Models

Text classification with LLMs

Integrating knowledge with language models

This class should be complemented by the following page with resources:
https://amazing-output-ccd.notion.site/Resources-for-LLMs-53c88cc0e3684968a0020534b503594f

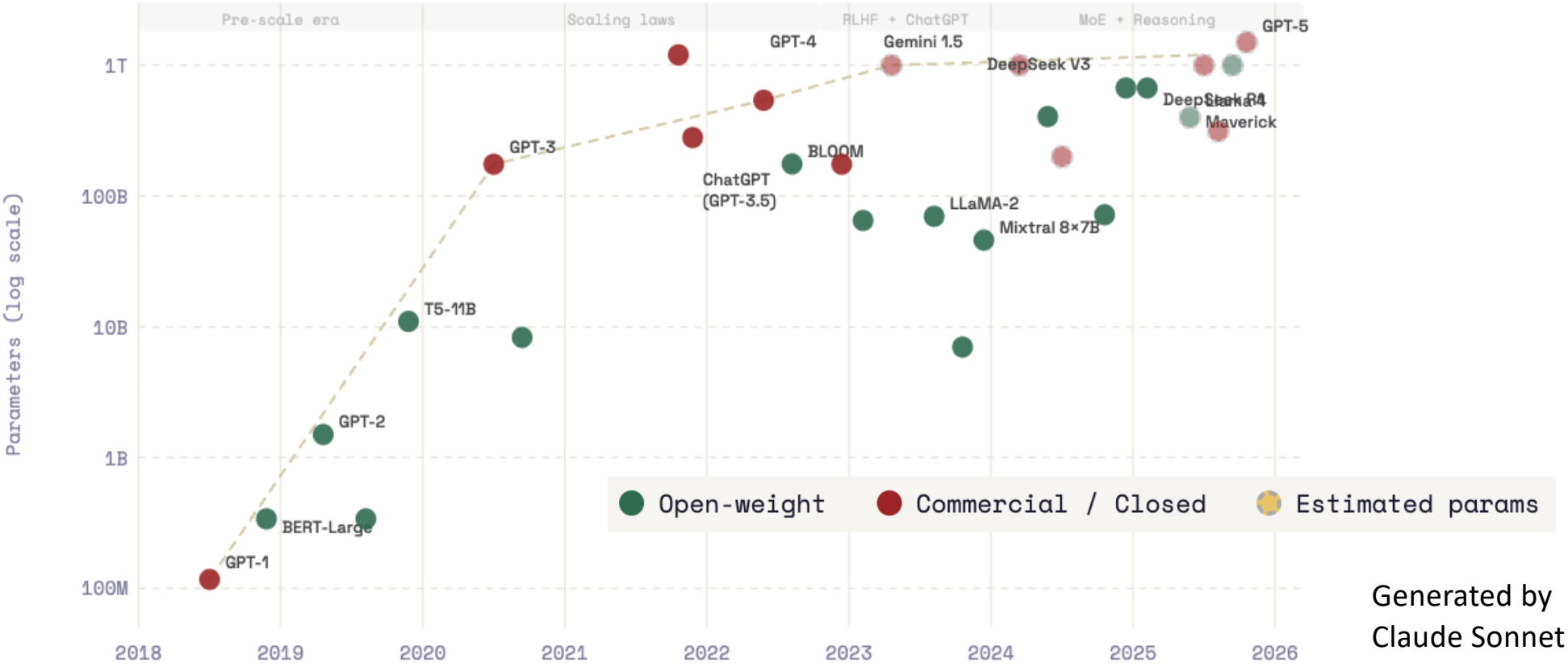
LLMs – main architectures

- **Encoder-decoder** – similar to the original transformers (e.g. T5)

- **Causal decoder** - unidirectional attention, allowing input tokens to only interact with those preceding them; the model generates the output sequence one token at a time, and at each step, it considers only the tokens that precede it (e.g. Llama, GPT)

- **Mixture of experts (MoE)** - the model is composed of multiple models, referred to as ”experts”, each of which specializes in some aspect of the data to enhance the overall model’s performance. Based on the idea of model ensembles in ML, is gaining traction (GPT4?, Mistral)

LLMs constantly evolving



Generated by
Claude Sonnet

An interesting example if you want to train your (mini) LLM (for Esperanto):
https://huggingface.co/blog/how-to-train

LLMs – “older” models

- **IT** – Instruction tuning
- **RLHF** – Reinforcement Learning with Human Feedback

List of some of the available LLMs
September 2024 (not updated)
From Paulo Seixal MSc thesis

Model	Release	Creators	Purpose	Size	Architecture Type	IT	RLHF	Availability
T5	Oct 2019	Google	General	11 B	Encoder-decoder	-	-	Open-source
CodeGen	Mar 2022	Salesforce	Code	16 B	Causal decoder	-	-	Open-source
GLM	Oct 2022	Multiple	General	130 B	Prefix decoder	-	-	Open-source
Flan-T5	Oct 2022	Google	General	11 B	Encoder-decoder	●	-	Open-source
BLOOM	Nov 2022	BigScience	General	176 B	Causal decoder	-	-	Open-source
Galactica	Nov 2022	Meta	Science	120 B	Causal decoder	-	-	Open-source
LLaMA	Feb 2023	Meta	General	65 B	Causal decoder	-	-	Open-source
CodeGen2	May 2023	Salesforce	Code	16 B	Causal decoder	-	-	Open-source
StarCoder	May 2023	BigCode	Code	15.5 B	Causal decoder	-	-	Open-source
LLaMA2	July 2023	Meta	General	70 B	Causal decoder	●	●	Open-source
Mixtral-8x7b	Dec 2023	Mistral	General	46.7 B	Mixture-of-experts	●	-	Open-source
LLaMA3.1	Jul 2024	Meta	General	405 B	Causal decoder	●	●	Open-source
GPT-3	May 2020	OpenAI	General	175 B	Causal decoder	-	-	Closed-source
Codex	July 2021	OpenAI	Code	12 B	Causal decoder	-	-	Closed-source
AlphaCode	Feb 2022	Google	Code	41 B	Encoder-decoder	-	-	Closed-source
InstructGPT	Mar 2022	OpenAI	General	175 B	Causal decoder	●	●	Closed-source
PaLM	Apr 2022	Google	General	540 B	Causal decoder	-	-	Closed-source
AlexaTM	Aug 2023	Amazon	General	20 B	Encoder-decoder	-	-	Closed-source
GPT-4	Mar 2023	OpenAI	General	1.7 T*	Mixture-of-experts*	●	●	Closed-source
PaLM2	May 2023	Google	General	340 B*	Causal decoder	●	-	Closed-source
Claude 2**	July 2023	Anthropic	General	137 B*	Causal decoder	●	●	Closed-source

LLMs – recent models

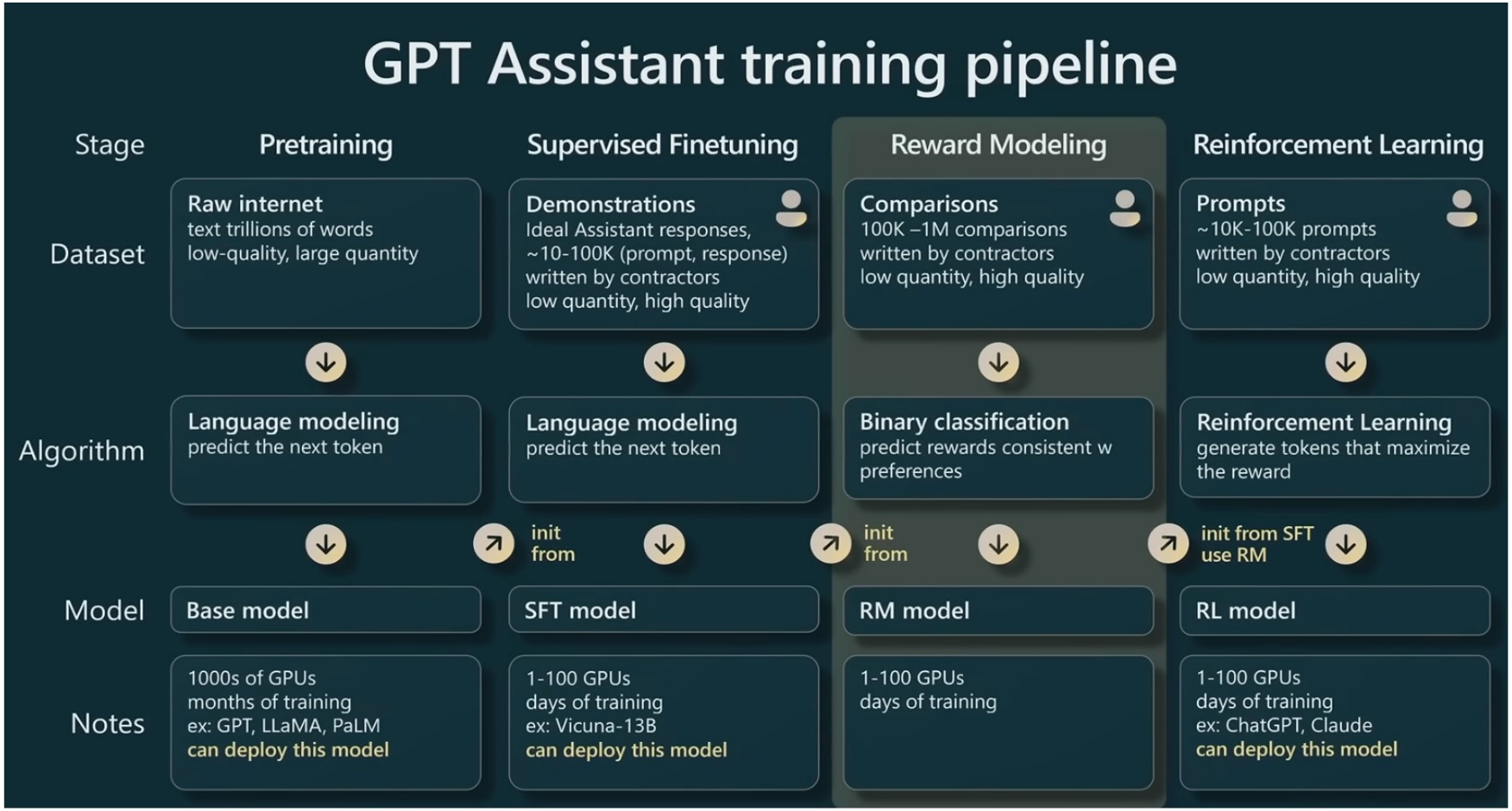
Model	Company	Parameters	Architecture	Availability
DeepSeek-V3	***DeepSeek-AI***	700B	MoE	Open
Llama 4	***Meta AI***	400B	MoE	Open
Mistral Large 3	***Mistral AI***	123B	MoE	Open
Qwen 3	***Alibaba***	235B	MoE	Open
Gemma 2	***Google***	27B	Dense transformer	Open
Phi4	***Microsoft***	16B	Dense transformer	Open
GPT5	***OpenAI***	2-5T ?	Dense transformer, Multimodal	Commercial
Claude Opus/ Sonnet	***Anthropic***	?	Dense transformer	Commercial
Gemini 2.5	***Google***	>1T ?	Dense transformer, Multimodal	Commercial
Grok 4	***xAI***	1.7T ?	Dense transformer	Commercial

LLMs – open-source - examples

- **BERT** (Google, 2018) - quite used in NLP; improved in last years; currently thousands of open-source, free, and pre-trained Bert models available for specific use cases
- **BLOOM** (2022) – 176B; 46 languages; 13 programming languages; access to source code and data
- **Llama3.1** (Meta, July 2024) – 8B, 70B, 405B; several languages; RLHF; chatbots + NLP
- **Mistral** (Sep 2023) – 7B; French company providing competitive models, some of reduced size; most recent Mistral3
- **OPT** (Meta, 2023) - 125 M to 175 B; similar performance to GPT-3
- **XGen-7B** (Salesforce, 2023) - 7B, 8x7B (mixture experts); supports longer context windows
- **DeepSeek** (2025)– several versions (R1, V3, etc)
- **Gemma** (Google, 2024) – most recent Gemma 2
- **Phi 4** (Microsoft, 2025) – 16B
- **Qwen 3** (Alibaba, 2025) – 235B

Model repo: https://huggingface.co/models

Training and tuning LLMs - stages



Andrej Karpathy

Training LLMs - Pre-training and fine-tuning

Most LLMs go through a **pre-training stage** to reach the foundational (general-purpose) models (e.g. GPT, Llama, etc)

Datasets include many different sources, as books, news, social media, scientific papers, code repositories, and several other websites

Ensures that the model gains exposure to various writing styles, terminologies, and contexts, enabling a good understanding of language.

Once the pre-training phase is completed, the model enters the **fine-tuning** phase - training on **specific datasets** tailored to the desired application or a specific domain

Instruction tuning

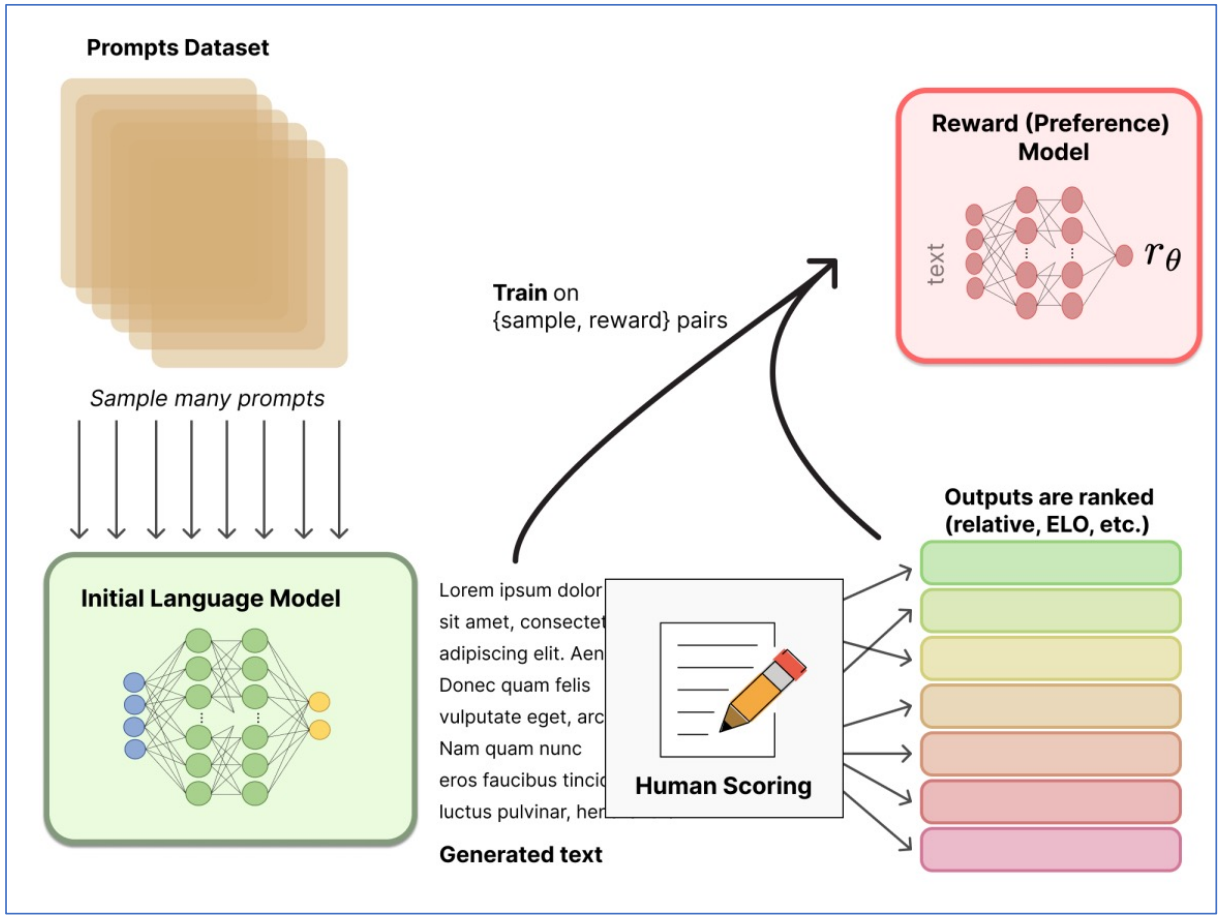
- **Supervised Fine-Tuning** (SFT) that serves as a bridge between a model's raw knowledge and its ability to act as a functional assistant – teaching the model to follow commands
- The LLM is trained in a **supervised way**: dataset comprising input-output pairs, where the input constitutes a natural language instruction, and the output corresponds to the desired response
- This approach guides the LLM to generate outputs that not only resemble the target responses, but also take into account the provided instructions, aligning the model more closely with specific task requirements
- Focuses on **correctness**. It is best when there is a "right answer" to a task

Reinforcement learning with human feedback

- RLHF was an addition to the AI methods toolkit, introduced from 2017 (Christiano et al, 2017), and widely used to refine LLMs for applications involving interactions with humans - **preference tuning**
- The idea is simple: after pre-training an LLM (decoder), it can generate different responses given a dialog (prompt). These can be evaluated by humans who assign numerical scores to the various options
- Alternative/ complement to IT - Instead of being given a "correct" answer, the model is trained to maximize a reward signal learned from human preferences
- These scores can be used to create rewards in the context of reinforcement learning algorithms

Paper: https://arxiv.org/abs/1706.03741

Reinforcement learning with human feedback



Process starts with a pre-trained model (e.g. GPT) that can be refined with human text (not shown in the image)

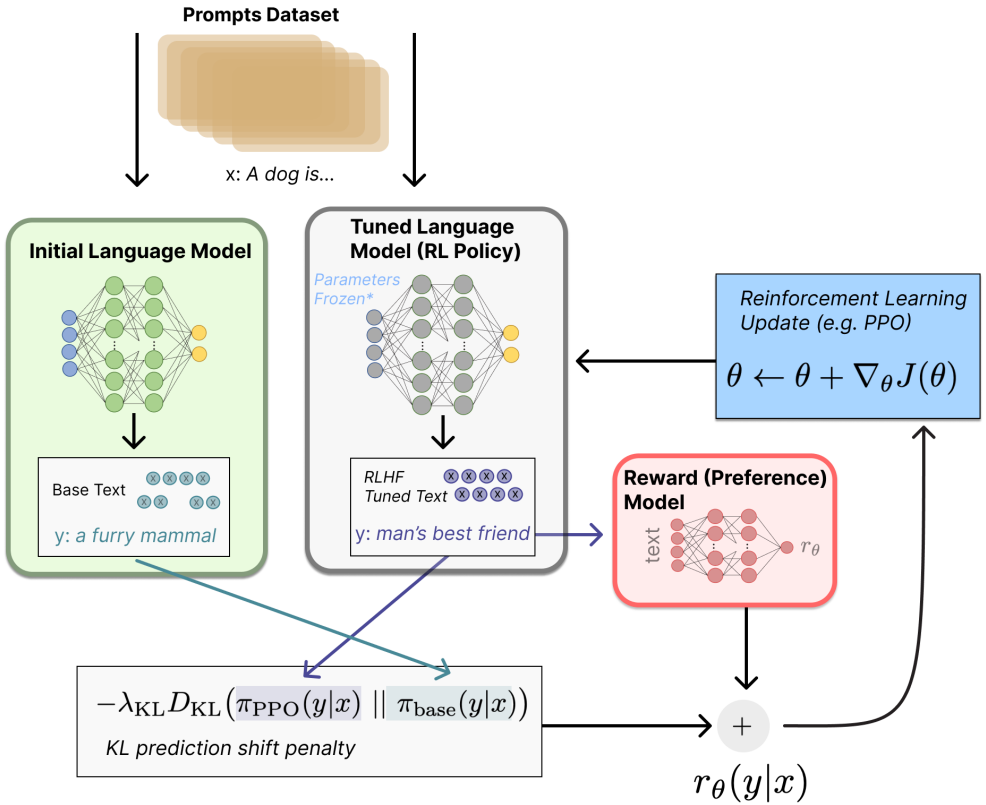
Reward (preference) model – trained to return a numeric value given a prompt and a response to this prompt
These models can be based on LLMs or trained from scratch
These models, once trained, can be introduced into the automatic model refinement pipelines

To generate data to train the reward model, prompts are provided to the original LLM that generates responses
Human annotators are used to sort these answers and create rankings.
These rankings are normalized and used to create numerical scores (e.g. using Elo systems)

Image and details: https://huggingface.co/blog/rlhf

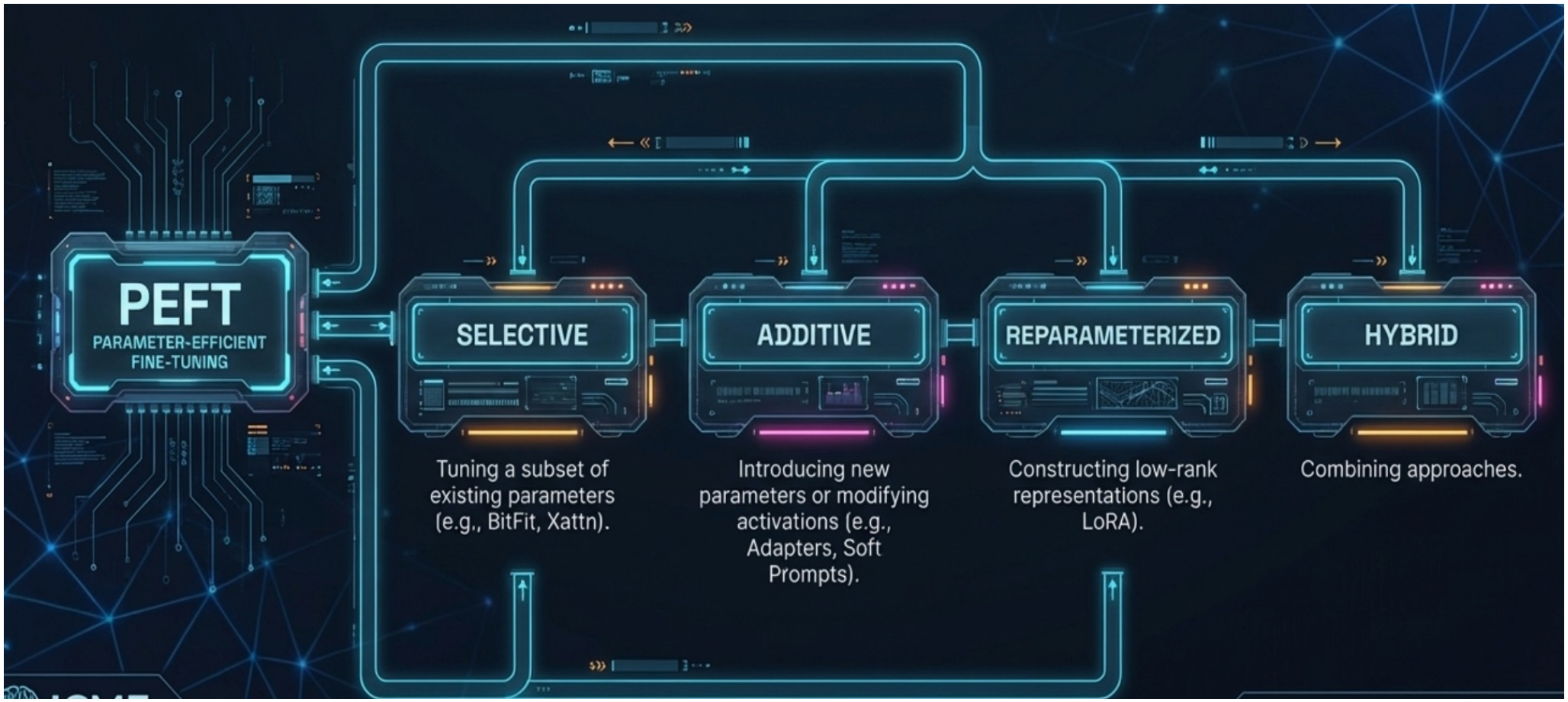
RLHF - optimization

- Once we have the reward model (RM), we can define the task as a reinforcement learning problem:
  - the policy will be implemented by the LLM that we want to refine
  - the reward will be given by the RM (loss will be the output of the RM for the outputs provided by the current LLM, subtracted of a penalty for outputs very different from the original LLM)
  - states given by the possible tokens and their responses.
- Multiple algorithms of Reinforcement Learning can be used, being the most common today Proximal Policy Optimization (PPO)



https://huggingface.co/blog/rlhf

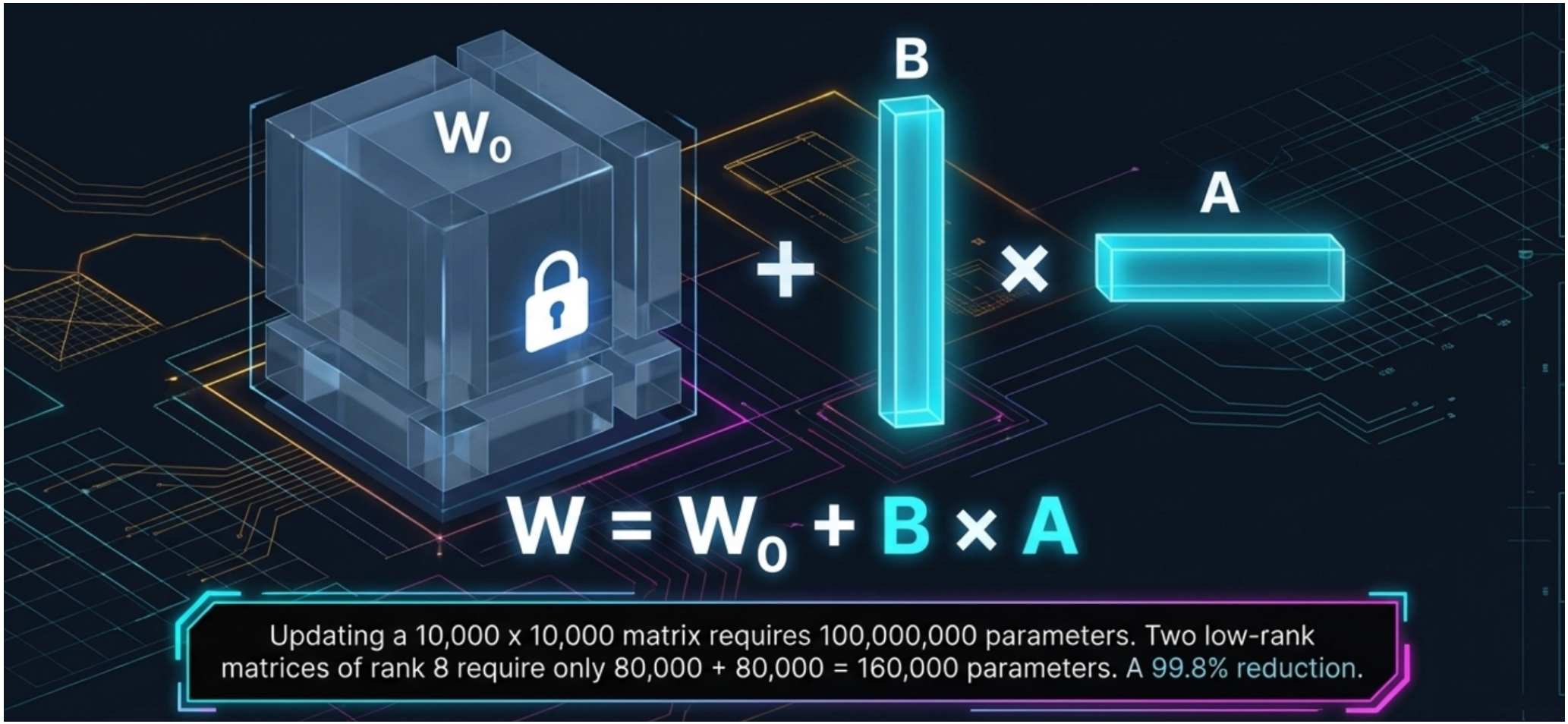
Parameter-efficient refinement



LLM tuning - LORA

- **LORA** (Low-Rank Adaptation): tuning only a fraction of parameters, freezing the rest of the model
- Instead of updating every parameter in the original weight matrix, LoRA introduces two low-rank trainable matrices that are trained in parallel, while the original model weights remain frozen
- This makes it parameter-efficient because it only requires training a small fraction of additional parameters to achieve performance comparable to full fine-tuning
- **QLoRA:** extension of LoRA designed to drastically reduce RAM usage during the refinement process. The primary difference is that QLoRA **quantizes** the frozen weights of the base model to a 4-bit precision format, to relieve memory bottlenecks

LORA illustration



LLM tuning - PEFT

- Beyond **LoRA**/ **QLoRA**, there are several other **Parameter-Efficient Fine-Tuning (PEFT)** methods
- **Additive models:** introduce **new trainable parameters or modules** into the existing architecture while keeping the original weights frozen
  - **Adapters:** insert small bottleneck layers within Transformer blocks, consisting of a down-projection matrix, a non-linear activation, and an up-projection matrix. Can be **Serial Adapters**, which are placed after attention or feed-forward layers, or **Parallel Adapters**, which run alongside the original layers.
  - **Soft Prompting (Prompt Tuning):** Instead of modifying weights, prepends adjustable, continuous vectors to the start of the input sequence in the embedding space
  - **$(IA)^3$:** introduces learnable **rescaling vectors** that modify the activations of the key, value, and FF components

LLM tuning - PEFT

- **Selective models:** focuses on **training a specific subset of the model's existing parameters**
  - **BitFit: only the bias terms** of the model are tuned
  - **Dift Pruning:** uses a learnable binary mask to determine which specific weights should be updated
  - **LayerNorm tuning:** achieves efficiency by adjusting **only the weights of the LayerNorm layer**
- **Hybrid Methods**: combine the strengths of different PEFT categories to create more robust fine-tuning frameworks
  - **UniPELT:** integrates **LoRA, prefix-tuning, and adapters** into a single block, using a gating mechanism
  - **MAM Adapter:** This combines **prefix-tuning** in the self-attention layers with **parallel adapters** in the feed-forward layers for optimal performance

LLM tuning - knowledge distilation

Transfer the learning of a pretrained model (teacher) to a **smaller student model**.

Tries to match the **outputs (logits) or internal representations** of the teacher model. The goal is for the student to achieve nearly identical performance to the teacher but with fewer resources

Distillation can be used to compress long prompts into a few special "**gist tokens**". The model is trained so that its behavior when using these short gist tokens matches its behavior when using the original long prompt

Compared to LoRA: While **LoRA** makes fine-tuning efficient by only updating a fraction of the parameters, **knowledge distillation** is a **performance-transfer** technique used to create compressed representations that "graduate" by mimicking the expertise of a larger foundation model

Using LLMs: applications

- Used directly in dialog applications (chatbots) or by using the APIs available to provide:
  - Direct answers to questions
  - Generation of texts given a prompt
  - Code Generation
  - Text enhancement: grammar correction, style, …
  - NLP tasks (zero-shot learning) – e.g. translation, summarization, recognition of entities/relationships, sentiment analysis, topic modeling...
  - Productivity enhancement
  - Multimodal application becoming more and more common
  - … and many others

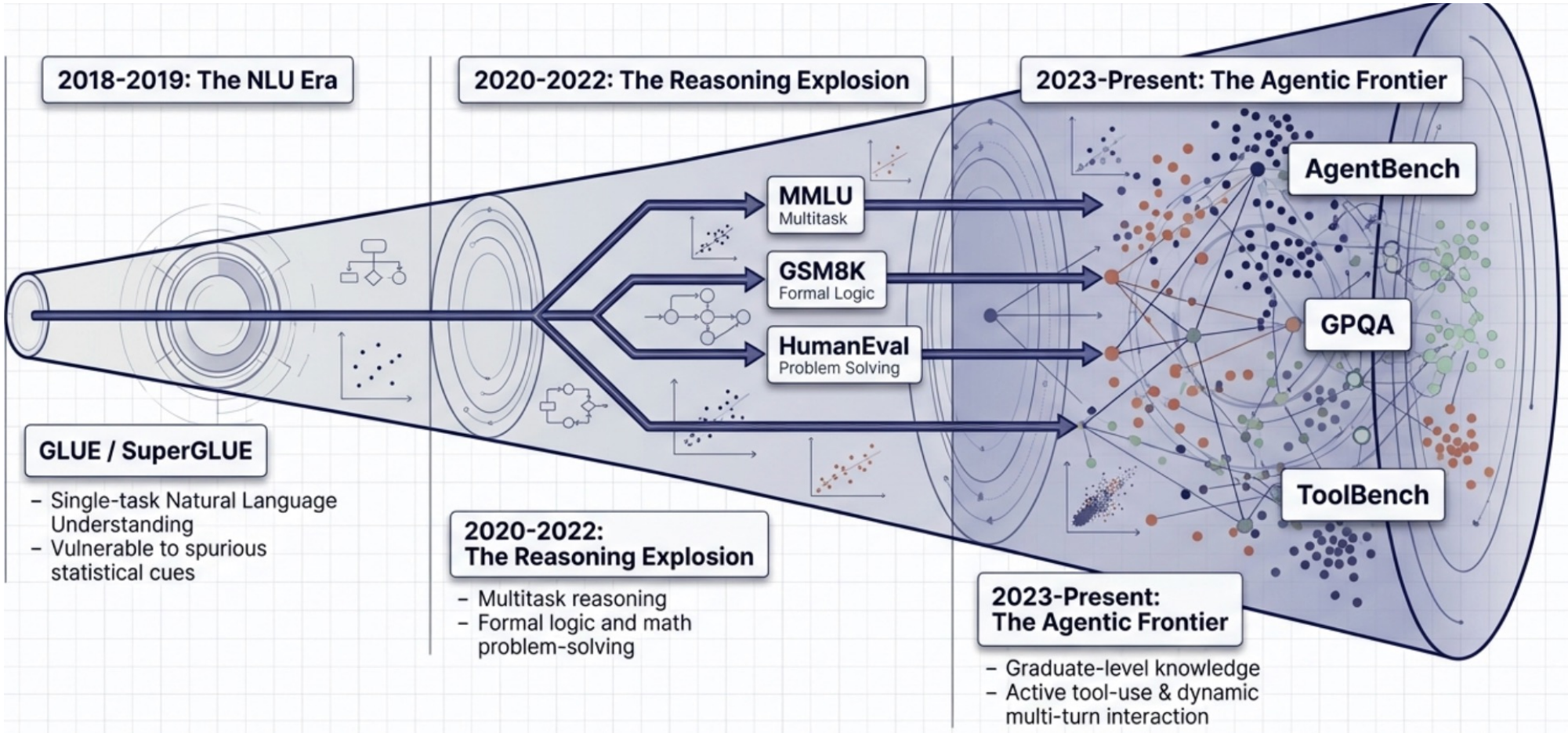
LLM assessment

- To assess the performance of LLMs, many methods have been proposed
- Two key components: **metrics** and **benchmarks**
- **Metrics** are quantitative measurements or standards for assessing the performance of LLMs providing specific numerical values, enabling the evaluation of different dimensions of a model's effectiveness, efficiency, or quality.
  - ML derived metrics - accuracy, precision, recall, F1 score
  - **Perplexity** – how well a training set has been learned
  - Recall-Oriented Understudy for Gisting Evaluation (**ROUGE**) or Bilingual Evaluation Understudy (**BLEU**) scores - more used in test sets

LLM assessment - benchmarks

- Benchmarks serve as standardized tasks, datasets, or challenges against which LLM performance is evaluated.
- Provide a common reference point for comparing different models or approaches, contributing to a consistent and reproducible evaluation of LLMs.
- LLM benchmarks encompass a wide range of tasks:
  - sentiment analysis
  - text summarization
  - question-answering, and many more

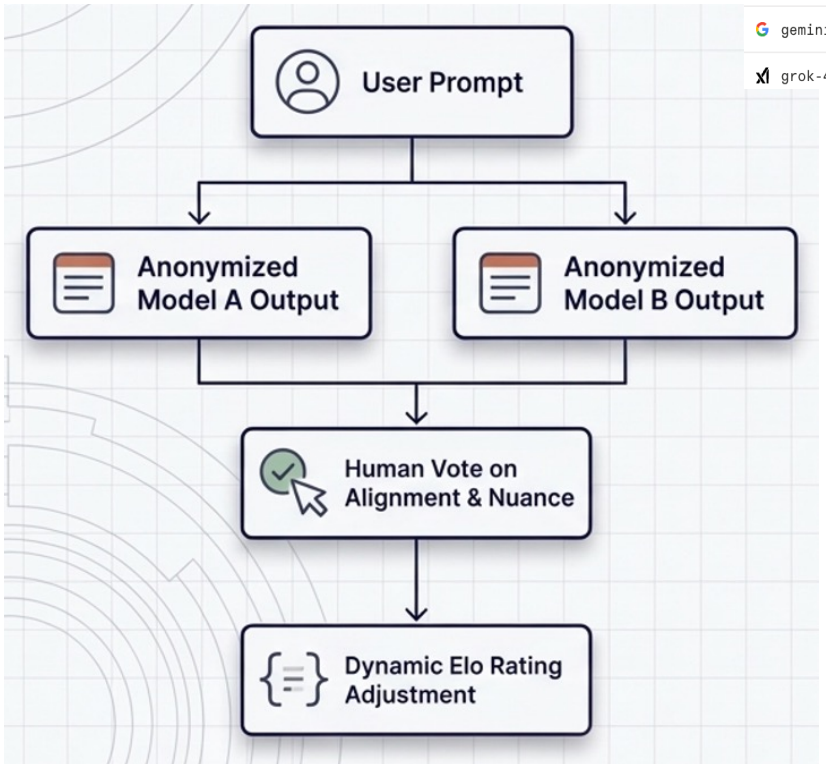
LLM assessment – evolution of benchmarks



Chatbot Arena

Blind A/B testing – human prompts and evaluation

Model 618 / 618	Overall	Expert	Hard Prompts	Coding	Math	Creative Writing	Instruction Following
claude-opus-4-6	1	1	2	3	3	5	2
claude-opus-4-6-t…	2	2	1	1	1	1	1
grok-4.20-beta1	3	13	4	6	16	2	8
gemini-3.1-pro-pr…	4	4	3	4	2	4	3
gemini-3-pro	5	8	6	10	6	3	9
gpt-5.4-high	6	3	5	2	4	6	5
gpt-5.2-chat-late…	7	9	7	7	5	10	10
gemini-3-flash	8	10	10	16	7	8	13
grok-4.1-thinking	9	19	13	18	25	19	28



https://huggingface.co/spaces/lmarena-ai/arena-leaderboard
https://arena.ai/
https://openlm.ai/chatbot-arena/

Assessment – novel dimensions

Risk Dimension	Primary Vulnerability	Diagnostic Benchmark	Metric of Failure
Safety & Alignment	Susceptibility to adversarial jailbreaks and prompt injection.	JailbreakBench, HarmBench	Attack Success Rate (ASR), Refusal Rate
Hallucination	Factual inconsistency and deviation from user context.	HaluEval, TruthfulQA	Faithfulness score, Factual fabrication rate
Data Leakage	Memorization and inadvertent disclosure of PII.	WikiMIA	Minimum-k% probability, Leakage rate
Robustness	Fragility to distributional shifts and prompt perturbations.	AdvGLUE, PromptRobust	Performance drop under adversarial attack

LLM as a judge

- Approach to evaluation where a high-performing model is used to assess the quality of outputs from other models
- Traditional automated metrics are often **inadequate** for evaluating sophisticated LLMs because they fail to capture semantic equivalence or the nuances of natural conversation
- LLM-as-a-judge allows for:
  - **Multi-faceted Profiling:** It can assess previously unquantifiable qualities, such as **coherence, creativity, and safety**.
  - **Context-Sensitive Evaluation:** Models can be assigned instance-specific criteria, such as "evaluate safety in this medical advice context"
  - **Scalability:** offers a way to perform large-scale open-ended evaluations that would otherwise be too expensive or time-consuming for human experts

Domain specific models

- LLM fine-tuning lead to the development of **domain-specific models** designed to understand specific nuances within a certain field using corpora of data specific to a given domain
- Advantage against general **LLMs** that cannot comprehend the same intricacies and specific echnical terms
- Domains:
  - **Natural Sciences (Mathematics, Physics, Chemistry, and Biology)** - abstract reasoning and symbolic manipulation through benchmarks like **ChemEval**, **ChemCrow**, **SciKnowEval, PubMedGPT, BioGPT**
  - **Engineering & Technology:** tools for **code generation**, **software engineering**, and technical tasks like **CAD script generation** (CADBench) or **aerospace manufacturing** (AeroMfg-QA)
  - **Education:** integration into **K-12 classroom scenarios**, supporting both student learning (problem-solving) and teacher-oriented tasks (automatic grading and material generation) through frameworks like **EduBench**
  - **Humanities & Social Sciences:** domains such as **Law** (legal support/ document generation- **LawBench**), **Finance** (stock prediction/ news analysis with **FinEval**), and **Psychology** (mental health support -**CPsyExam**)
  - **Healthcare** – e.g. Med-Palm-2 , Almanac
  - And many many others…

Prompt Engineering – what is it?

- A **prompt** serves as the fundamental unit used to interact with a LLM, varying from simple inputs or instructions to complex templates, guiding it to generate relevant and contextually appropriate outputs to accomplish a given task.
- **Prompt engineering** - field that involves optimizing prompts for efficient use of LLMs for a set of applications
- Why?
  - Better results when using LLMs
  - Helps to test LLMs and their capabilities and limitations
  - Allows you to build innovative applications on top of LLMs' APIs
- Can be used both through **direct interaction** with chatbots or using a predefined **API** access to a LLM

Resources: https://www.promptingguide.ai/

Prompt engineering – elements of the prompt

Instruction: core element that describes the specific task the model should perform, such as "Summarize this text," "Write a short story," or "Rewrite this sentence using passive voice".

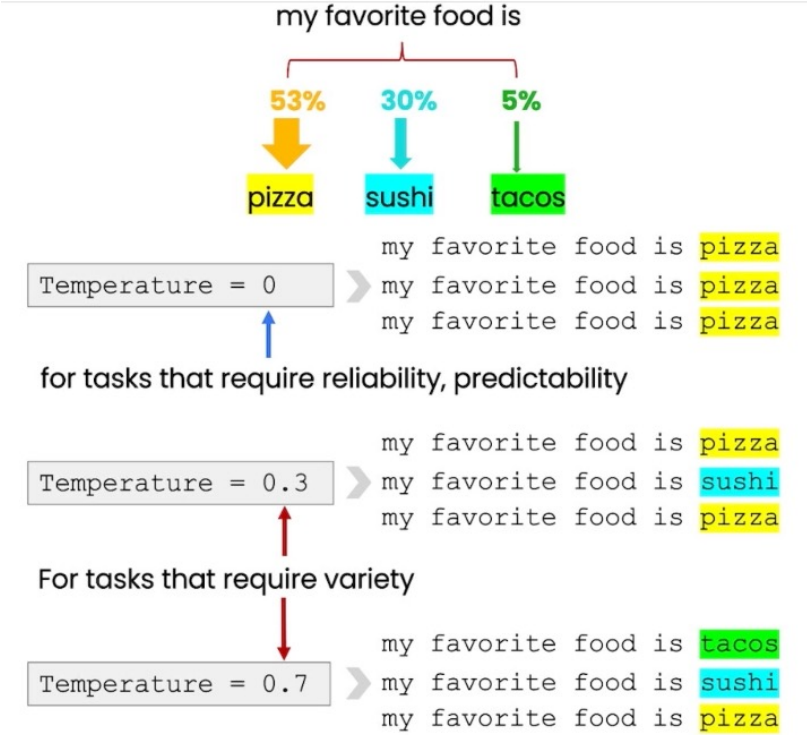
Context and Examples: Prompts often include background information or specific examples to guide the model.

Prompt Templates: models are often trained using standardized templates. A classic "Alpaca-style" template includes a prefix that signals a task is beginning (e.g., "Below is an instruction that describes a task...") followed by the user's specific instruction.

Input Data: This is the specific material that the instruction acts upon, such as the actual text to be summarized or the code to be corrected

Prompt engineering – parameters/ settings

- The most relevant parameter in the APIs of LLMs – ***temperature*** – allows to control stochasticity of the responses
- It is a value that determines the probability of choosing the most likely token at each step of generation
- Higher values favor greater diversity in responses
- Lower values should be used for greater predictability



https://learn.deeplearning.ai/chatgpt-prompt-eng
Other settings: https://www.promptingguide.ai/introduction/settings

Prompt Engineering – principles

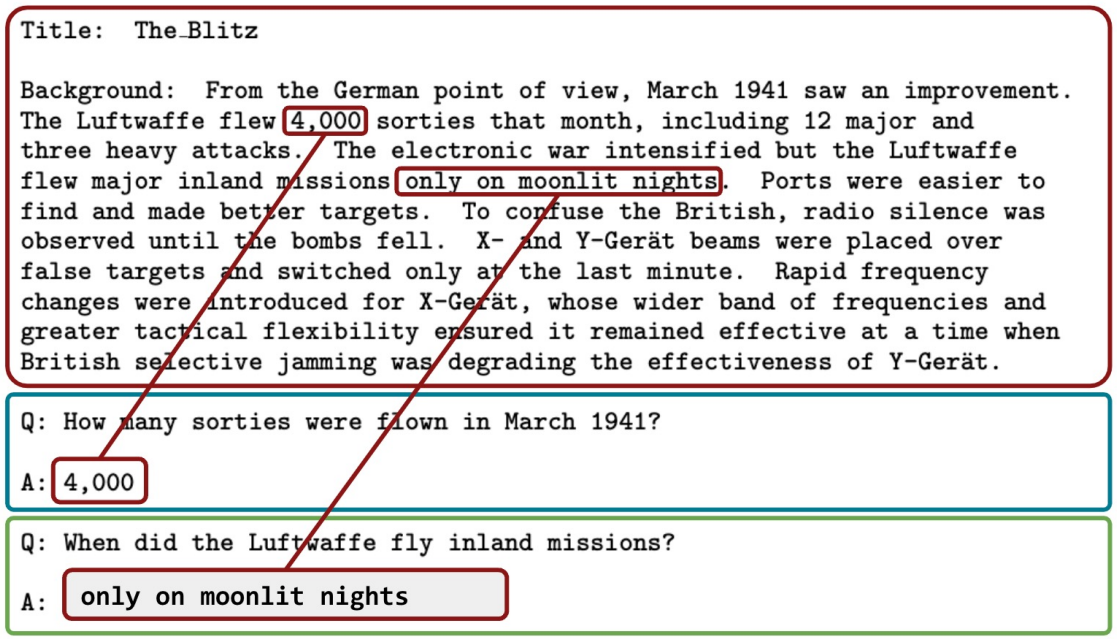
- Write **clear** sentences that are as unambiguous and **specific** as possible
- Various types of separators can be used to give **structure** to requests when they become more complex
  - Structure can be requested in the output (e.g. JSON ou HTML)
- **Iterative process** - try what you want, analyze the result and clarify the instructions by improving the context
- In complex tasks, explicitly ask the model to reason step by step before giving a final answer
- Use examples in context to clarify what you want, including positive and negative instructions to show what is mean or not

Course: https://learn.deeplearning.ai/chatgpt-prompt-eng

Using LLMs: context

- The ability to do **"in-context"** learning can be used in LLMs to improve the outcomes of prompts, feeding the model with texts that can **create an appropriate context for the task or questions that follow**
- Can be used to perform one- or few-shot learning, giving examples to the LLM that allow "generalization"
- It can be used to give "instructions" on how to perform some tasks, how to organize answers, etc
- Can be used to change the text generation style, for example in “role playing”

In-context learning in LLMs



LLMs allow you to create a context that can then be used for subsequent dialogs and tasks to ask the LLM

Adapted from Christopher Potts – Stanford

Zero-shot learning

- Large LLMs are tuned to follow instructions and are trained on large amounts of data; so they are capable of performing some tasks "zero-shot”
- This means **no further learning** is needed to solve some problems/ tasks
- Example:

> *Prompt*: Classify the text into neutral, negative or positive.
> *Text*: I think the vacation is okay.
> *Sentiment*: Neutral

- Note that in the prompt above we didn't provide the model with any examples of text alongside their classifications, the LLM already understands "sentiment” from previous training.
- Instruction tuning has shown to improve zero-shot learning - Wei et al. (2022) - the concept of finetuning models on datasets described via instructions.

Zero-shot learning example with context

Context

On the next set of prompts, I will provide you with an ID followed by a paragraph that has been written by either a human or an AI system. Please just indicate if it was the first case (human written) using the format ID:Human or the second (AI-generated) using the format ID:AI

Got it! Send the IDs and paragraphs, and I'll classify them accordingly.

Input

ID1-1 -The cell cycle, or cell-division cycle, is the sequential series of events that take place in a cell that causes it to divide into two daughter cells. These events include the growth of the cell, duplication of its DNA (DNA replication) and some of its organelles, and subsequently the partitioning of its cytoplasm, chromosomes and other components into two daughter cells in a process called cell division. In eukaryotic cells (having a cell nucleus) including animal, plant, fungal, and protist cells, the cell cycle is divided into two main stages: interphase, and the M phase that includes mitosis and cytokinesis.

ID1-1: AI

Generated in Mar 19 2025 by Chat GPT

Zero-shot learning example

Input

D1-19 - There seems to be some merit to having a single daily glass of red wine, it seems to have something called resveratrol, which acts as an antioxidant and is thought to be good for your heart. Additionally, Italian researchers found that moderate beer drinkers had a 42 percent lower risk of heart disease compared to non-drinkers. For maximum protection, keep your consumption to one pint—at around 5 percent alcohol by volume—a day, the researchers say. Basically, alcohol, in moderation, appears to lower risks of heart and other cardiovascular disease.But this is in moderation, mind you. Just as with everything else, the only difference between medicine and poison is the dosage. A beer a day is good; twelve is not.

ID1-19: Human

Generated in Mar 19 2025 by Chat GPT

An example of one-shot learning

- **Prompt**:

A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word whatpu is:

We were traveling in Africa and we saw these very cute whatpus.

To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word farduddle is:

- **Output**:

When we won the game, we all started to farduddle in celebration.

We can observe that the model has somehow learned how to perform the task by providing it with just one example (i.e., 1-shot). For more difficult tasks, we can experiment with increasing the demonstrations (e.g., 3-shot, 5-shot, …)

One-shot learning example

Generated in Mar 19 2025 by Chat GPT

Now, let us keep the same task, but I will provide an example of each case:

Got it! Please provide the examples, and I'll classify the following texts accordingly.

First example - Human - D1-17 The recommended number of eggs to consume per week varies depending on several factors, including individual health, dietary preferences, and overall dietary patterns. It's important to note that while eggs are a nutritious food, they are also relatively high in dietary cholesterol. According to general guidelines, including those provided by the American Heart Association and the Dietary Guidelines for Americans, consuming up to seven eggs per week is considered reasonable and can be part of a healthy diet for most people. However, it's important to consider the overall context of your diet, including other sources of cholesterol and saturated fat, as well as your individual health goals and any specific dietary restrictions or considerations you may have.

Understood! Please provide the next text, and I'll classify it accordingly.

Second example - AI - D1-18 The question of how many eggs one should eat per day is often debated due to concerns about cholesterol and cardiovascular health. Current research suggests that consuming up to three eggs per day can be part of a healthy diet for most people, particularly young, healthy adults. Consuming up to three eggs per day has been shown to increase HDL (good) cholesterol and improve the LDL/HDL ratio, which are favorable changes for cardiovascular health. For most healthy individuals, consuming up to three eggs per day can be beneficial, improving cholesterol profiles and providing essential nutrients without increasing cardiovascular risk. However, individuals with specific health conditions, such as iovascular disease or diabetes, should consider their overall dietary patterns.

One-shot learning example

Generated in Mar 19 2025 by Chat GPT

D1-22 Artificial sweeteners are low- or zero-calorie sugar substitutes used to sweeten foods and beverages without the added calories of sugar. Common examples include aspartame, sucralose, saccharin, and stevia (a natural alternative). They are widely used in diet sodas, sugar-free gum, and diabetic-friendly products. While they provide sweetness without spiking blood sugar levels, they remain controversial. Studies show they are safe in moderation, but concerns exist about potential long-term effects on gut health and metabolism. Some people experience sensitivity to specific sweeteners. Ultimately, artificial sweeteners can be helpful for reducing calorie intake, but their impact varies widely from person to person.

D1-22: AI

D1-25 Although the earliest life seems to have arisen quite soon in Earth's history, less than 800 million years after its formation, simple forms continued for 1.6 billion years until the multicellular Francevillian biota developed and quickly died out about 2.1 billion years ago. After that, although multicellular forms were in existence from about 1.7 billion years ago, it was not until the Cambrian 'explosion' from 540–500 million years ago that multicellular life expanded and prospered. This is essentially what we know. But fossilisation is rare and early life has resulted in tiny traces difficult to interpret. At present there are believed to be about one trillion species on Earth, less than one per cent of those which have ever lived.

D1-25: Human

Few-shot learning

- ***Few-shot prompting*** can be used as a technique to enable in-context learning where we provide demonstrations in the prompt to steer the model to better performance.
- A few more tips:
  - the label space and the distribution of the input text specified by the demonstrations are both important
  - the format you use also plays a key role in performance
  - selecting random labels from a true distribution of labels (instead of a uniform distribution) helps.

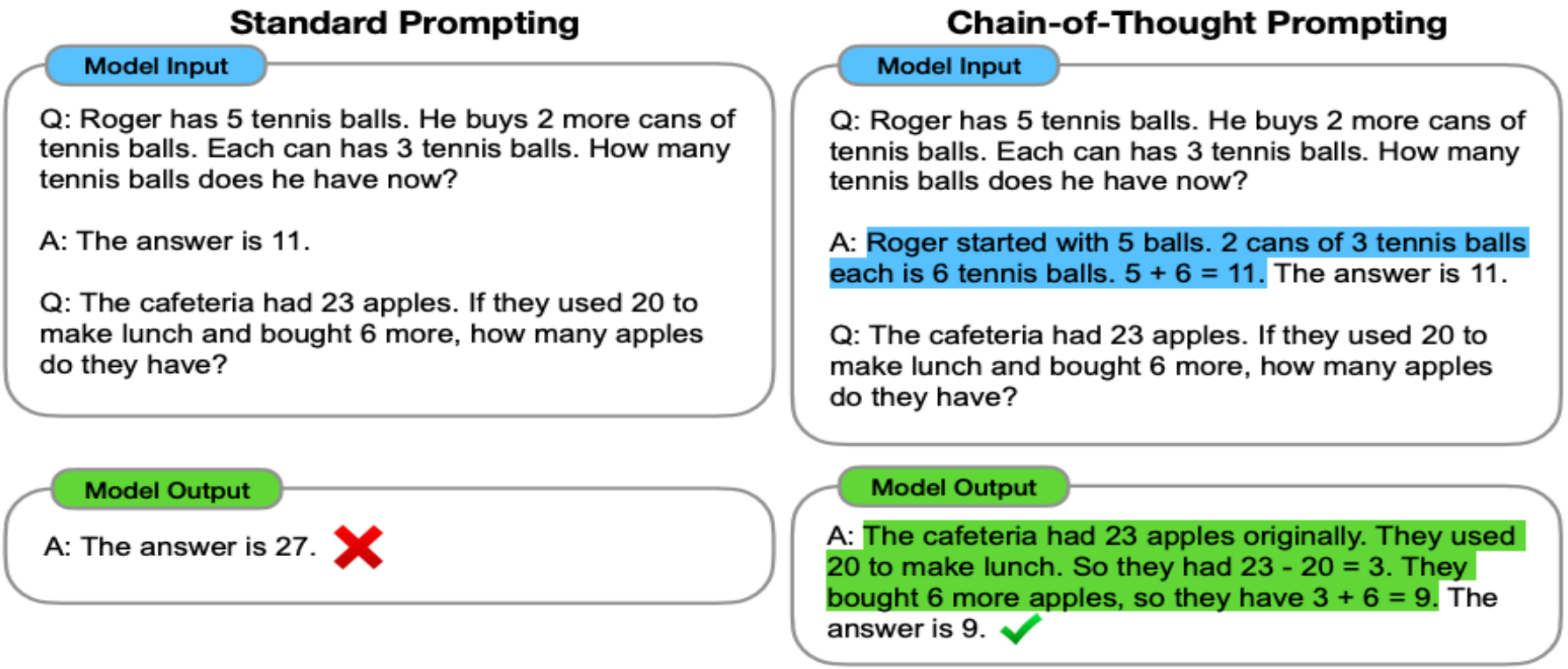
- **Prompt**:

This is awesome! // Negative
This is bad! // Positive
Wow that movie was rad! // Positive
What a horrible show!

- **Output**:

Negative

Chain of thought



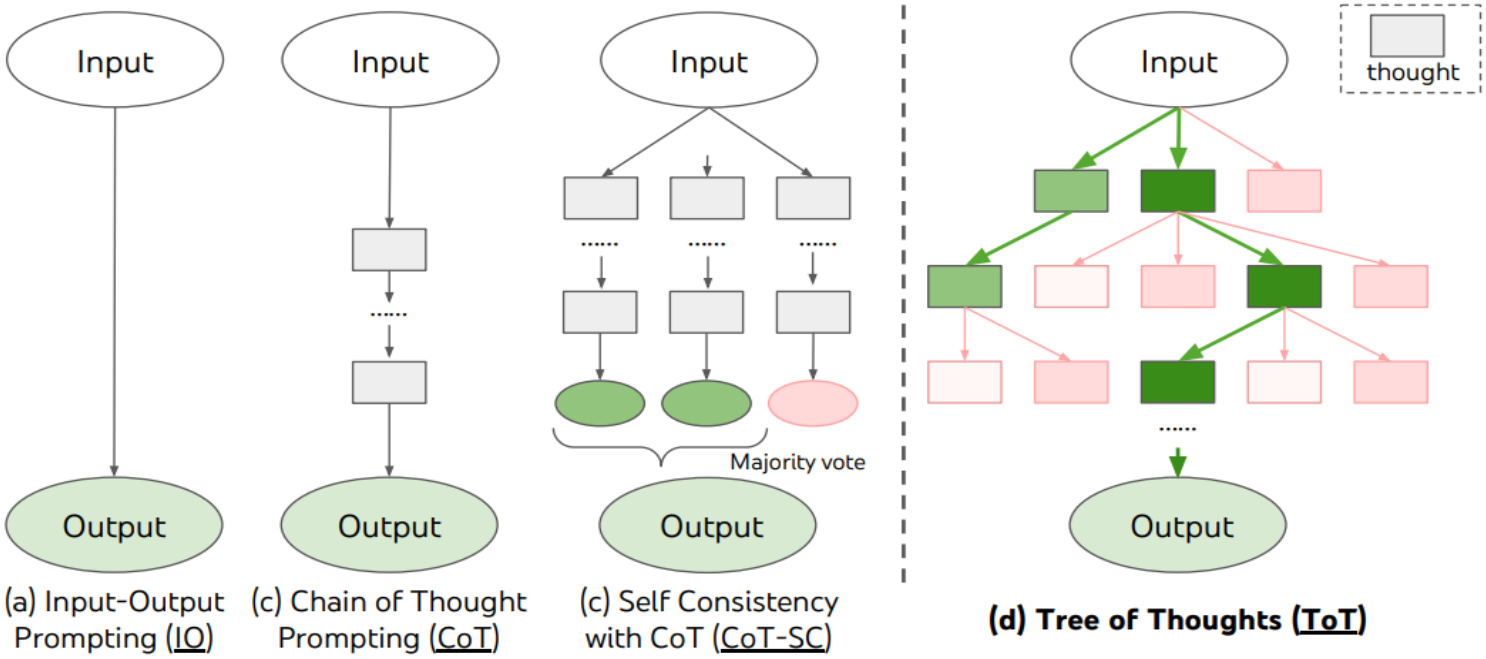
Chain-of-thought (CoT) prompting enables complex reasoning capabilities through intermediate reasoning steps – it is nowadays included in many chatbots

Prompt chaining

- To improve the reliability and performance of LLMs, one of the important prompting engineering techniques is to break tasks into its **subtasks**.
- Once subtasks have been identified, the LLM is prompted with a subtask and then its response is used as input to another prompt- **prompt chaining**
- Useful to accomplish complex tasks, where prompts perform transformations or additional processes on the responses before reaching a final desired state.
- Helps to boost transparency of your LLM application, increases controllability, and reliability.
- Particularly useful when building LLM-powered conversational assistants and improving the personalization and user experience of your applications.

Tree of thoughts

- For complex tasks, traditional or simple prompting techniques fall short. Alternatives have been proposed including Tree of Thoughts (ToT), a framework that generalizes chain-of-thought prompting and encourages exploration over thoughts that serve as intermediate steps.



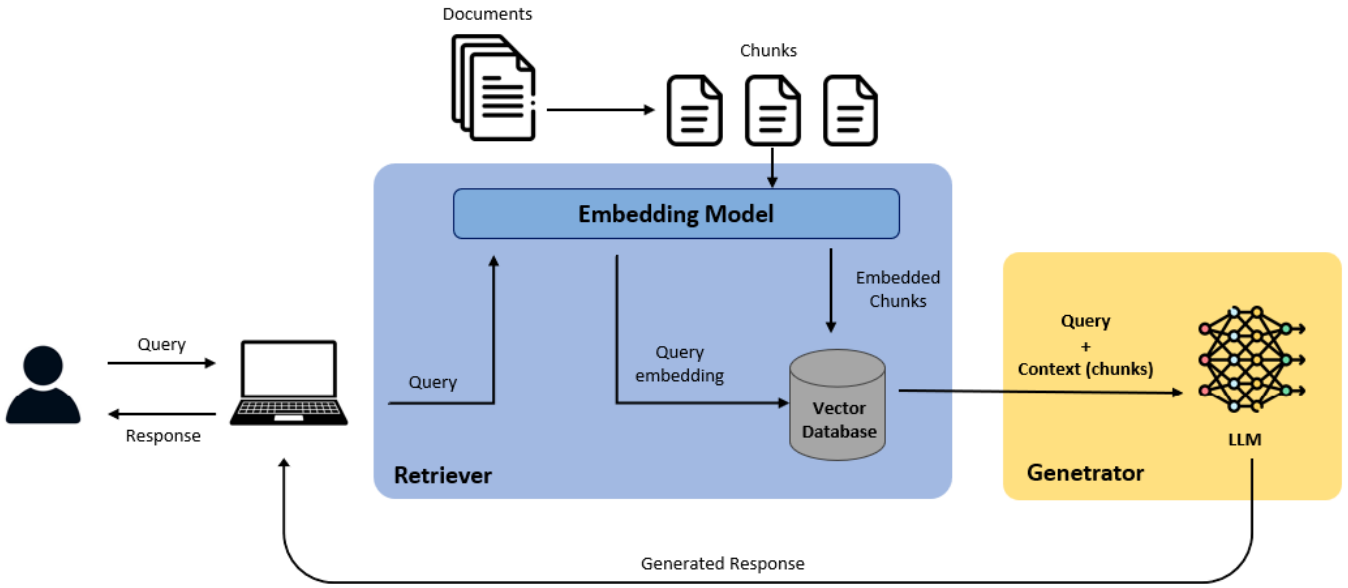
https://www.promptingguide.ai/techniques/tot

Retrieval Augmented Generation (RAG)

- For complex and knowledge-intensive tasks, it's possible to build an LLM-based system that accesses external knowledge sources to complete tasks.
- This enables more factual consistency, improves reliability of the generated responses, and helps to mitigate the problem of "hallucination".
- RAG can be finetuned and its internal knowledge can be modified in an efficient manner **without needing retraining of the entire model**.
- This makes RAG adaptive for situations where facts could evolve over time, enabling access to the latest information.

Retrieval Augmented Generation (RAG)

- RAG combines an **information retrieval component** with a **text generator model**.
- RAG takes an input and retrieves a set of relevant/supporting documents given a source.
- The documents are provided as context with the original input prompt and fed to the text generator which produces the final output.



Retrieval Augmented Generation (RAG)

- Step 1 – **Database construction:**

Creation of a vectorized database to store documents. Due to LLM's context window limits and to improve the similarity search, documents are segmented into manageable sections, transformed into numerical vectors and efficiently stored for rapid retrieval

- Step 2 – **Querying**

Similarly to the documents in the external knowledge base, user queries undergo vectorization through the same embedding model.

- Step 3 – **Information retrieval**

The retriever component conducts a search of the database using the vectorized query, identifying closely related sections within the documents. This process relies on calculating similarity

- Step 4 – **Response generation**

Following the retrieval of the most relevant sections, the prompt is assembled by combining the initial query with these retrieved sections as context. This prompt is then input into the LLM, seeking a coherent and contextually appropriate response to the user's query

Vector databases for RAG

- Vector databases, or vector stores, are specialized databases designed to store and retrieve data efficiently as high-dimensional numerical vectors.
- These allow accurate semantic similarity search based on vector similarity
- Allows to handle complex/ unstructured data
- Can be implemented to allow improved performance - Creating a robust vector database is crucial for the performance of a **RAG** system
- Steps in the construction:
  - Chunking - split the documents into smaller units
  - Converting textual data into numerical vectors – embedding
  - Indexing the numerical vectors for good performance

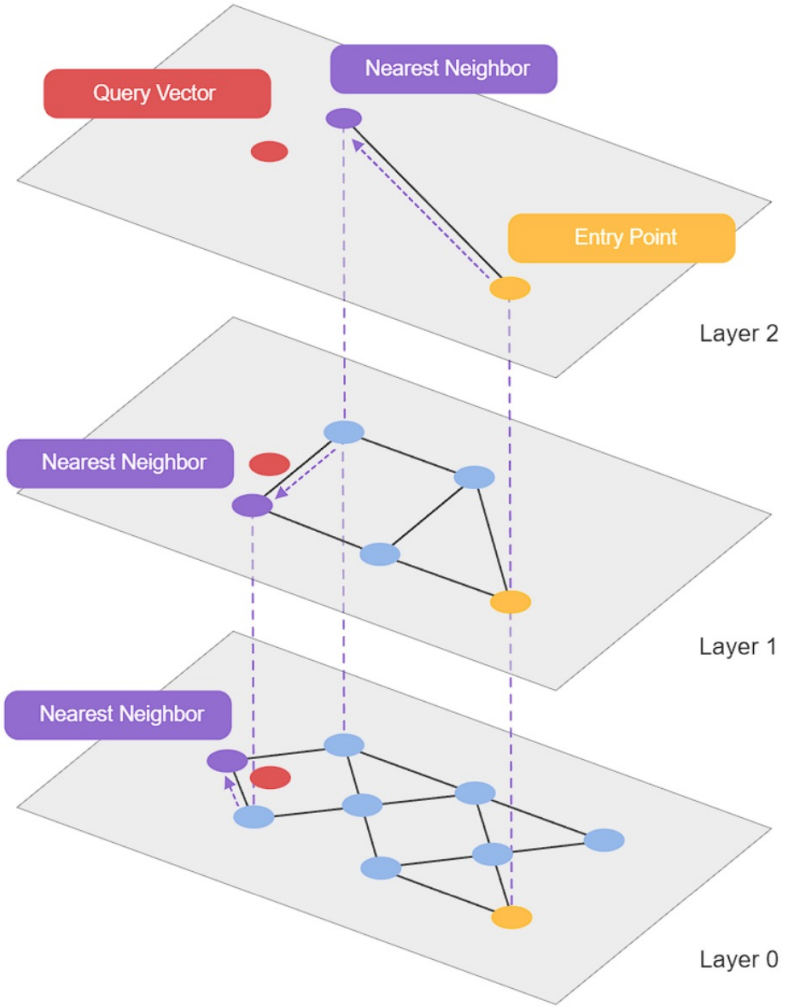
Similarity search/ retrieval algorithms for RAG

- **Nearest Neighbor Search - NNS**
  - seeks to find the vectors in a database that are closest to a specified query vector
  - operates based on distance metrics, such as Euclidean distance or cosine similarity, measuring the similarity between vectors.
  - common search algorithms include brute-force approaches and tree-based approaches
  - can have high computational complexity
- **Approximate NNS**
  - provide an approximate solution, relaxing the requirement for an exact match and reducing computational costs
  - One popular alternative is the use of Navigable Small World (NSW) graphs that introduce a trade-off between accuracy and efficiency
  - **Hierarchical NSW** further refines the structure. It introduces a hierarchy of layers, each containing a subset of nodes. Nodes within a layer are connected to nodes in the same and neighboring layers, creating a navigable structure

To learn more about this: https://www.deeplearning.ai/short-courses/building-applications-vector-databases/

Similarity search/ retrieval algorithms for RAG

HNSW

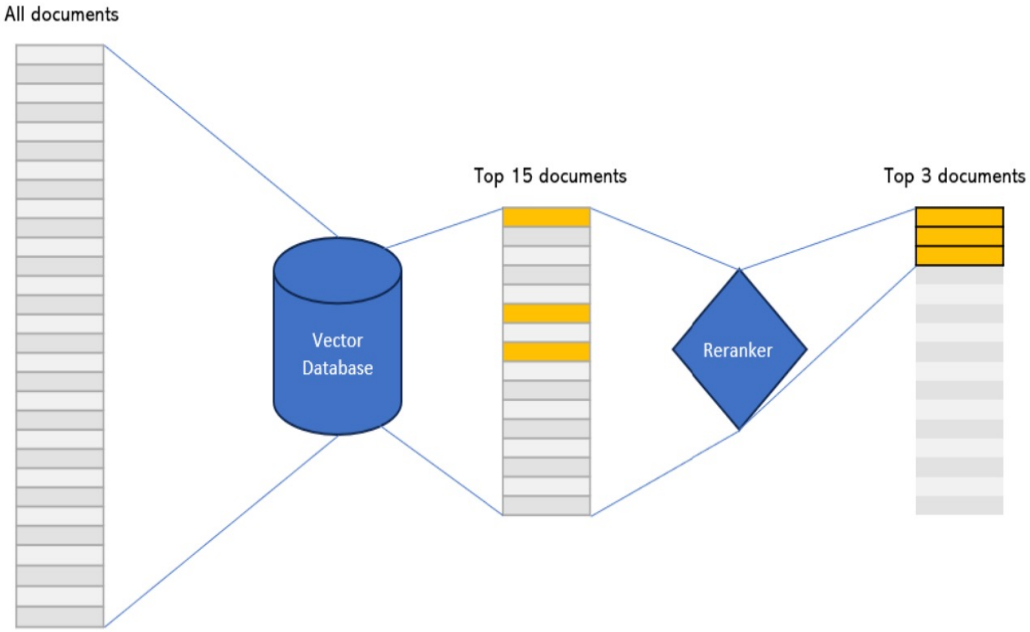


To learn more about this: https://www.deeplearning.ai/short-courses/building-applications-vector-databases/

RAG - rerankers

Some information loss occurs as the data is compressed into a singular vector - not uncommon that the top-ranked documents in search (e.g., the top three documents) may overlook pertinent information.

Addressing this issue may involve expanding the set of retrieved documents in the search and subsequently implementing a **reranking process**.



This is typically done by **cross-encoders**, a model that, given a query and document pair, will output a similarity score.

RAG - assessment

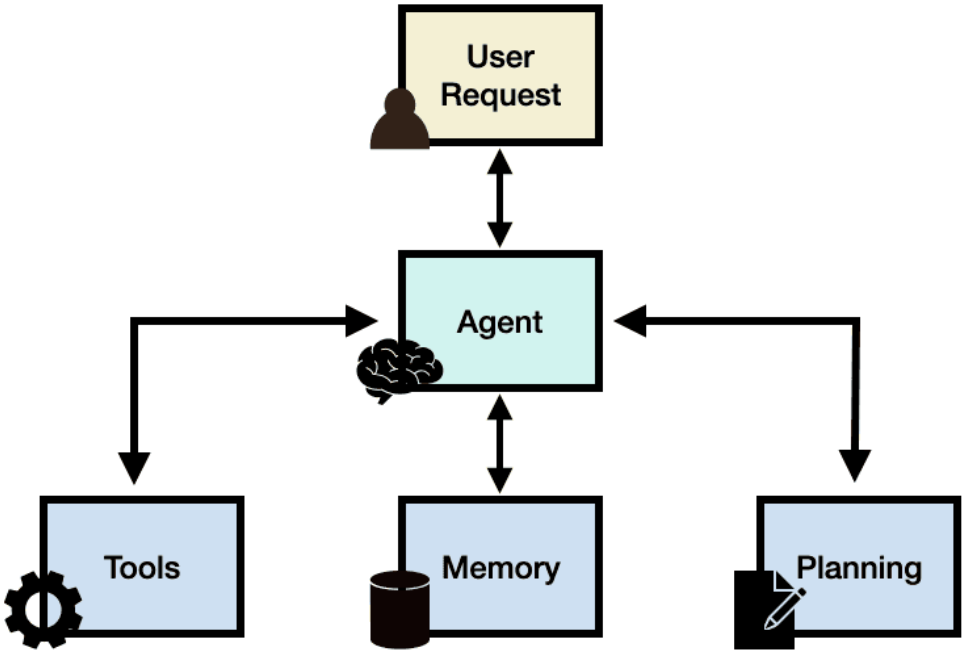
- Need to assess both the retriever and the generator
- Evaluating the retriever involves checking the ability to find relevant information from external sources- recall, precision, relevancy
- Generator can be evaluated by the capacity to create consistent and relevant responses, given the context
- Important to have quantitative metrics with defined cases - many times involves manual curation of prompts
- Some relevant metrics:
  - faithfulness (accuracy of the generated answer against the retrieved context)
  - answer relevance (if the generated answer addresses the question that was provided)
  - context relevancy and recall (recovers all relevant context minimizing redundancy)
  - answer semantic similarity (to the ground truth)

Relevant frameworks for RAG evaluation: RAGAS and ARES:

- Ragas: Automated evaluation of retrieval augmented generation. ArXiv, abs/2309.15217, 2023.
- Ares: An automated evaluation framework for retrieval-augmented generation systems. ArXiv, abs/2311.09476, 2023.

Agents - framework

- LLM agents seek to give the ability to integrate LLMs with external tools and services.
- When building LLM-based agents, the LLM serves as the central controller, or ”brain”, orchestrating a sequence of operations necessary to fulfil a task or user request.
- This central role involves interpreting the user’s input and determining the appropriate actions/tools to be executed



- When integrated with domain-specific tools, these agents can perform complex operations beyond their native capabilities, providing deeper insights, more accurate predictions, and customized solutions.

Levels of LLM agency

- **Lowest** - *Simple processor*: LLM output has no impact on program flow.

```
process_llm_output(llm_response)
```

- **Low** - *Router*: LLM output determines basic control flow.

```
if llm_decision(): path_a()
else: path_b()
```

- **Medium** - *Tool call*: LLM output determines function execution

```
run_function(llm_chosen_tool,
llm_chosen_args)
```

- **High** - *Multi-step Agent*: LLM output controls iteration and program continuation.

```
while llm_should_continue():
        execute_next_step()
```

- **High** - *Multi-Agent*: One agentic workflow can start another agentic workflow

```
if llm_trigger():
        execute_agent()
```

Agents - architecture

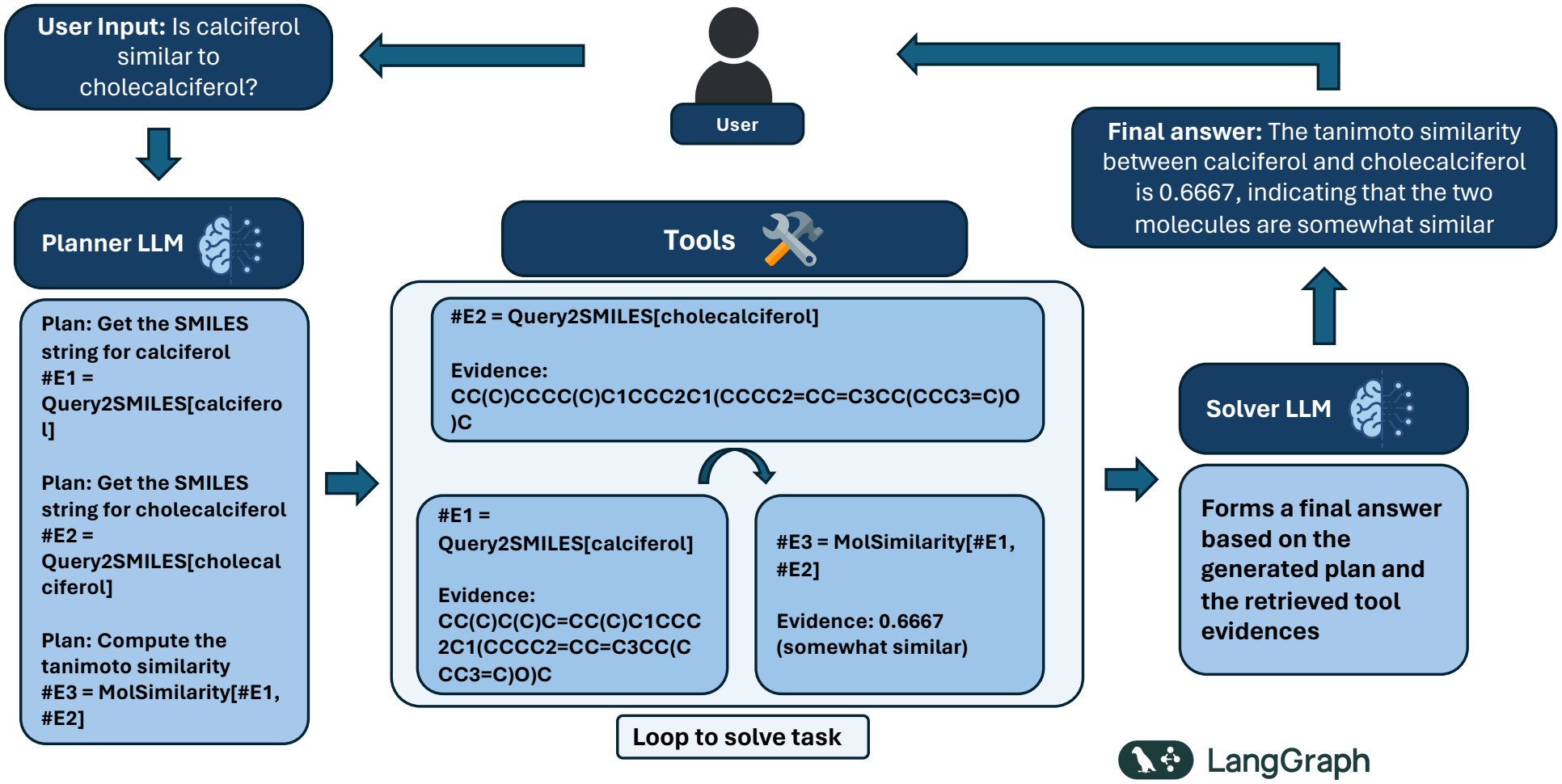
- To solve complex tasks, humans typically deconstruct them into subtasks and tackle them individually.
- Inspired by this human capability, most agent architecture designs aim to empower agents with a similar ability to break down and systematically address complex tasks.
- This enables the agent to create a sequence of actions or decisions, ensuring that each step contributes to the completion of the overall task.
- Existing studies on agent architecture design can be summarized on the basis of whether the agent can receive **feedback** during the planning process.

Agents - planning

Planning without feedback:

- LLM operates without receiving input from external tools after executing them. Consequently, the outputs of these tools cannot shape future decision-making processes, potentially leading to less adaptive or suboptimal strategies.
- **Reasoning without Observation (REWOO)** - separates plans from external observations, where agents first generate plans and obtain observations independently, and then combine them to derive the final results
- Planning without feedback may present some obstacles. In many real-world scenarios, agents must make long-term planning to solve complex tasks.

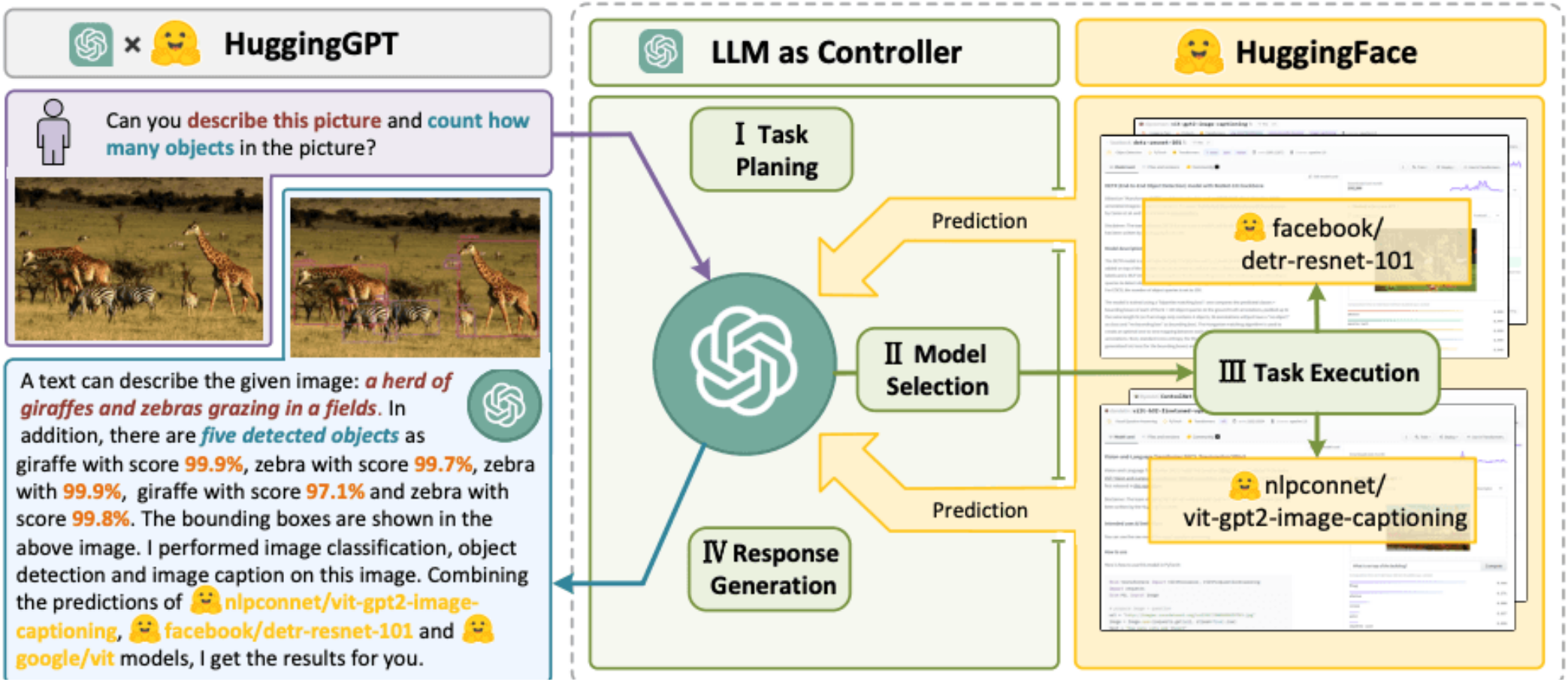
Agents – an example with REWOO



LangGraph

Paulo Seixal – MSc thesis - 2024

HuggingGPT: task planner

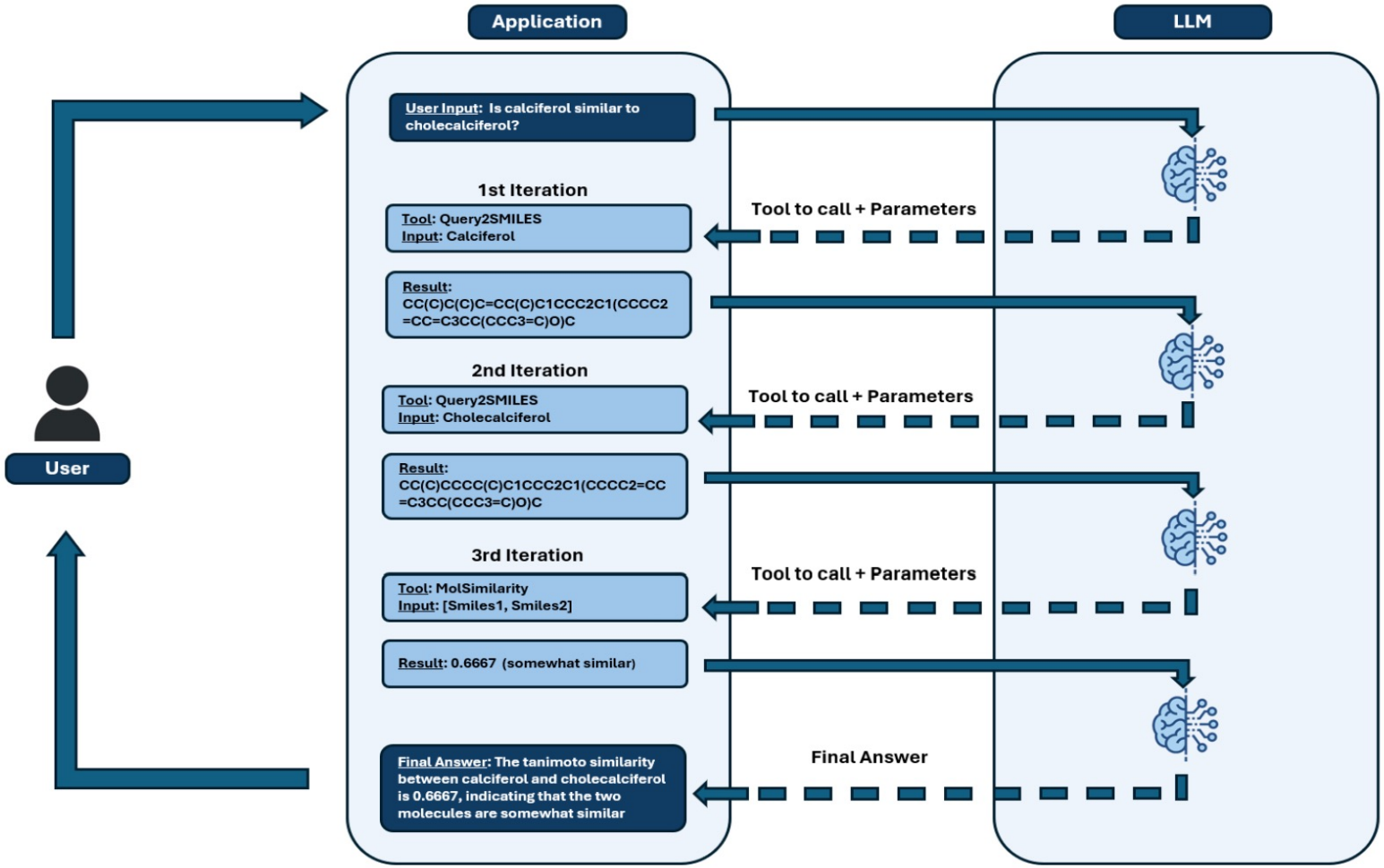


Agents - planning

Planning with feedback:

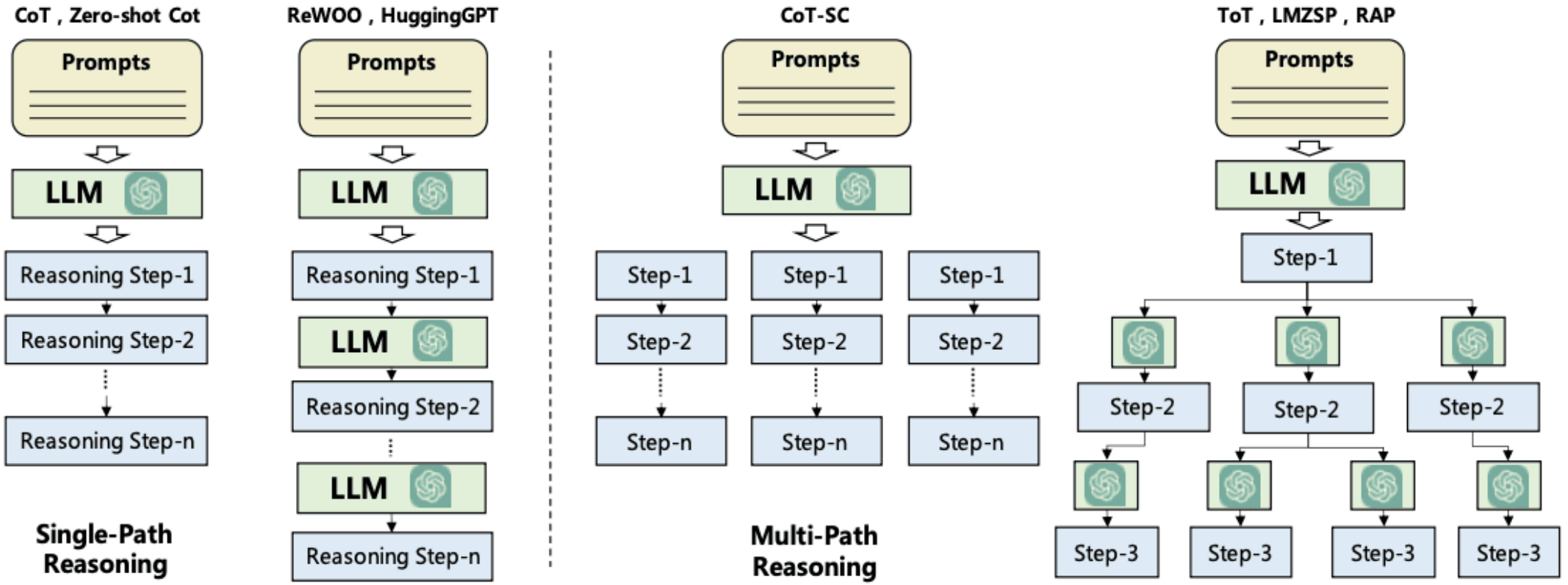
- Enables LLM-based agents to adapt and improve their actions based on external inputs and signals. This can be an advantage, especially when examining how humans tackle complex tasks, iteratively making and changing their plans based on external feedback
- **Reasoning and acting (ReAct)** - operates on a cycle of action generation, and the respective observation/result for that action. Initially, the LLM chooses an action to take based on its understanding of the task and context. These actions are then executed, and feedback is collected from the action result itself
- The feedback signals are used to refine the agent's understanding and decision-making process. The agent analyzes the feedback, identifies areas for improvement, and adjusts its action generation strategy accordingly. This iterative cycle continues, allowing the agent to continuously learn and optimize its actions over time

Agents – an example with ReAct



Paulo Seixal – MSc thesis - 2024

Comparing strategies



Wang et al, 2023

Reflexion

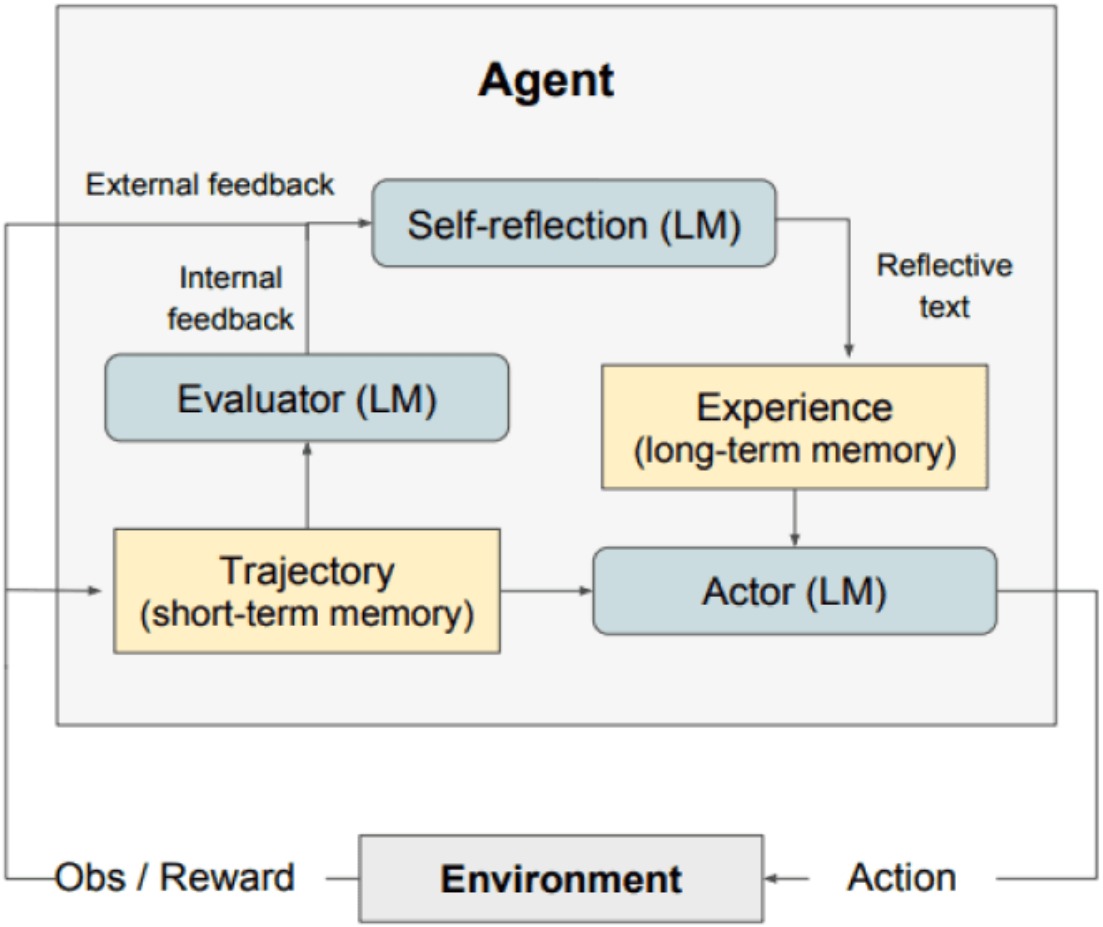
Simple Reinforcement Learning setup

Converts feedback from the environment into linguistic feedback, also referred to as **self-reflection**, which is provided as context for an LLM agent in the next episode

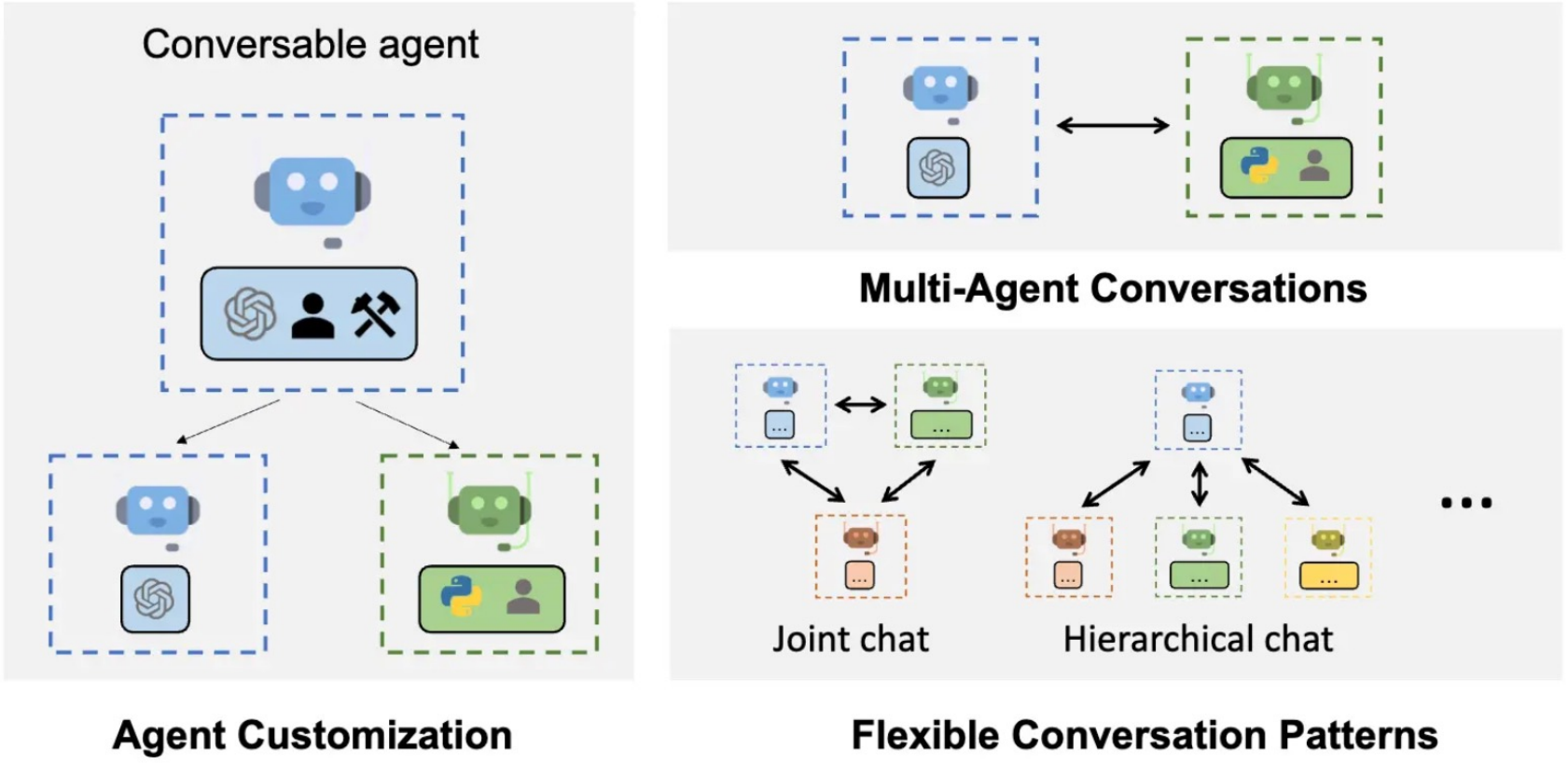
Actor: Generates text and actions based on the state. Takes an action in an environment and receives an observation. Chain-of-Thought (CoT) and ReAct are used as Actor models.

Evaluator: Scores outputs produced by the Actor.

Self-Reflection: Generates verbal reinforcement cues to assist the Actor in self-improvement.

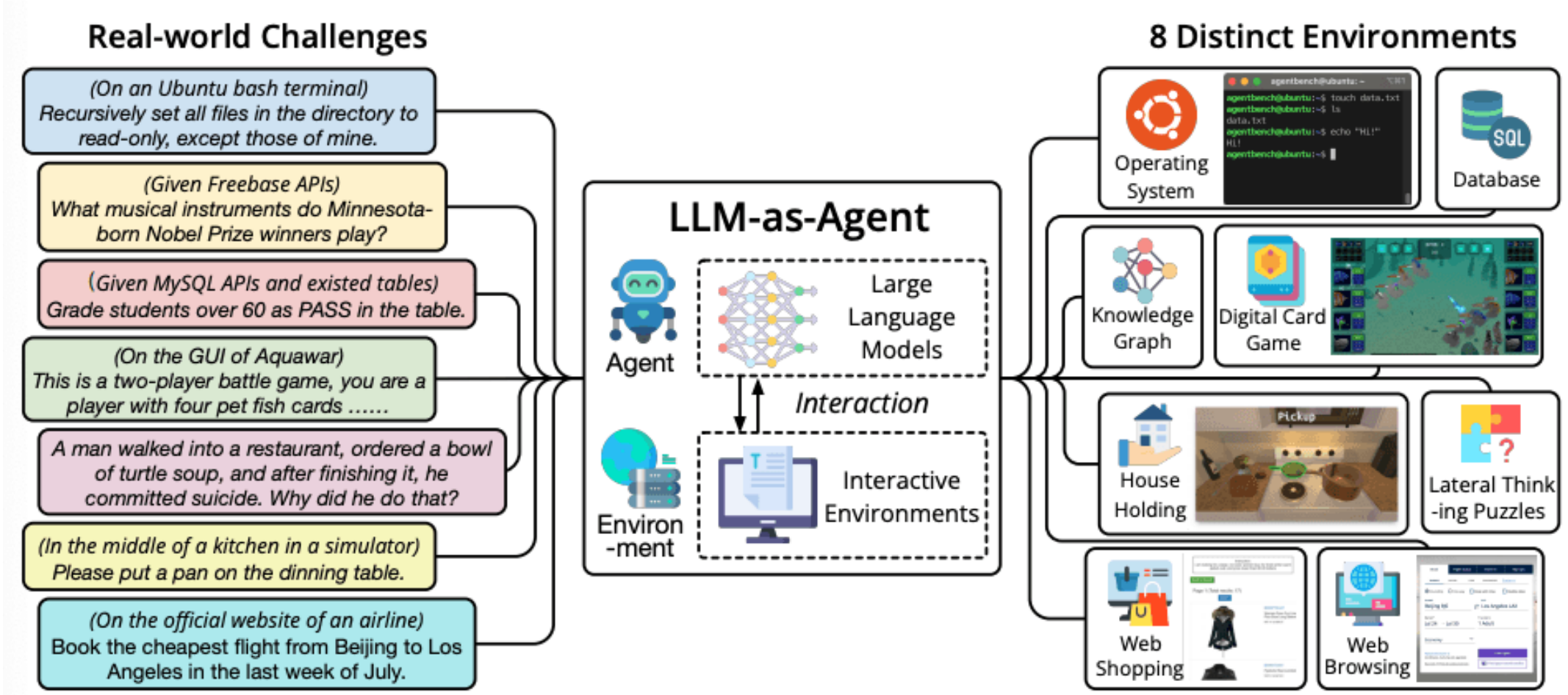


Agent and multi-agent organization

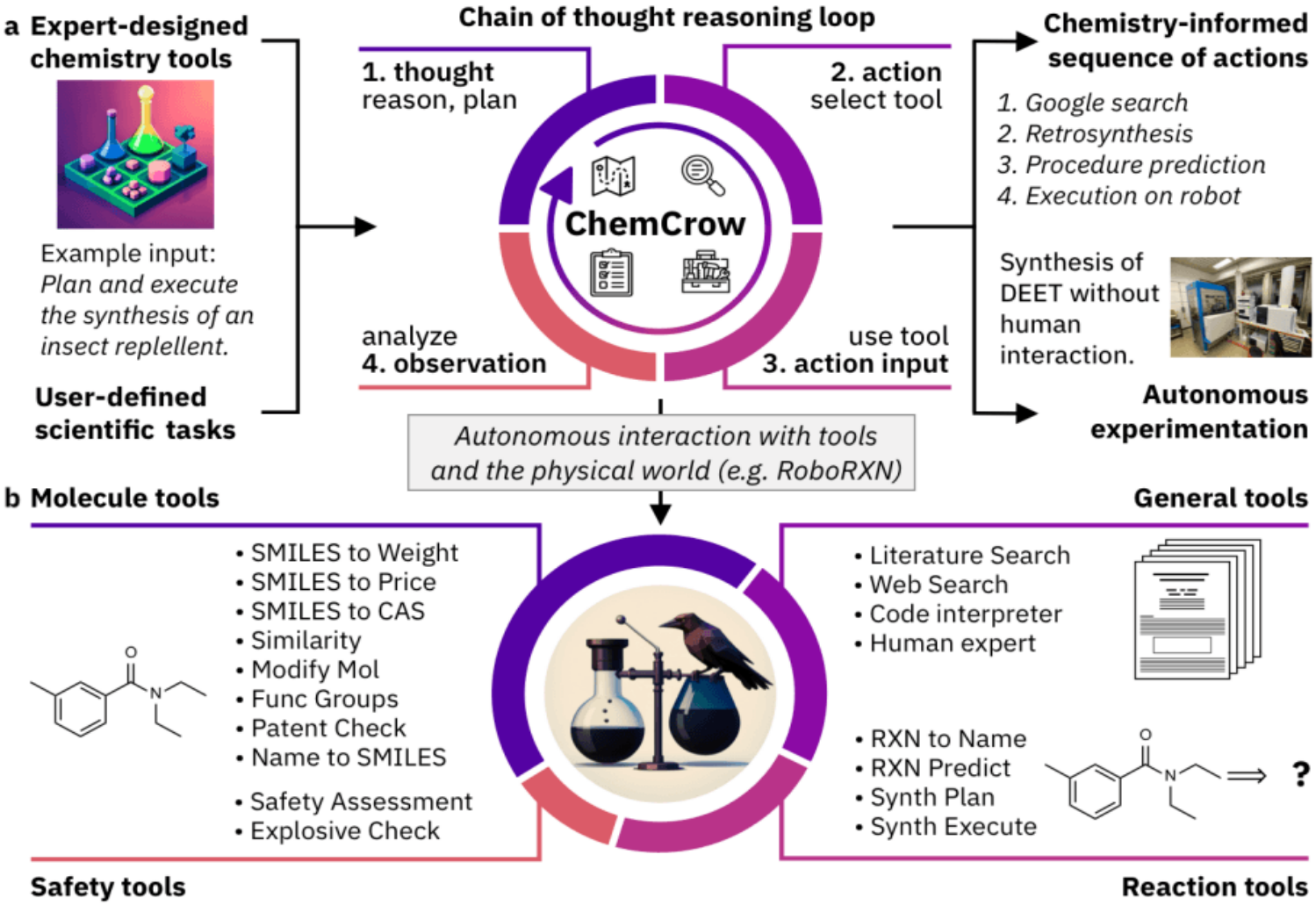


AutoGen capabilities; Figure Source: https://microsoft.github.io/autogen

Agents evaluation

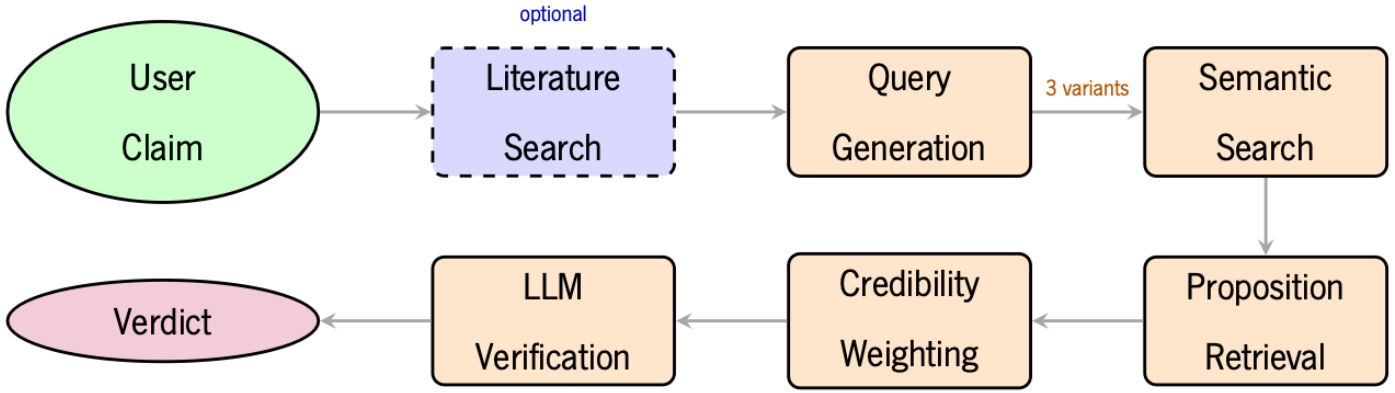


Example: ChemCrow



Application example: claim checking

- MSc thesis in Informatics Eng – Mateus Pereira – 2026
- **Aim**: design a system that can evaluate the scientific credibility of a given claim using mainly LLMs



https://github.com/Exemocaro/Scientific-Claim-Verifier

Claim checking – examples of prompts

Proposition evaluator

```
Please evaluate the following proposition based on the criteria below.
↪  Pay special attention to detecting vague or meta-commentary
↪  statements.

EVALUATION CRITERIA:
- **Accuracy**: Rate from 1-10 based on how well the proposition
↪  reflects the original text.
- **Clarity**: Rate from 1-10 based on how easy it is to understand the
↪  proposition without additional context.
- **Completeness**: Rate from 1-10 based on whether the proposition
↪  includes necessary details (e.g., dates, qualifiers, specific
↪  entities).
- **Conciseness**: Rate from 1-10 based on whether the proposition is
↪  concise without losing important information.

SPECIAL PENALTIES FOR VAGUE STATEMENTS:
If the proposition contains any of these characteristics, give it LOW
↪  scores (1-3):
- Meta-commentary about the document itself ("The study found...",
↪  "Results showed...", "The paper discusses...")
- Vague qualifiers without specifics ("significant results", "important
↪  findings", "key insights")
- Process descriptions without concrete facts ("analysis was conducted",
↪  "data was collected")
- Generic conclusions ("implications were discussed", "trends were
↪  observed")

Examples:

Original Text: "In 1969, Neil Armstrong became the first person to walk
↪  on the Moon during the Apollo 11 mission."

Good Proposition: "Neil Armstrong became the first person to walk on the
↪  Moon in 1969."
Evaluation: accuracy: 10, clarity: 10, completeness: 10, conciseness: 10

Bad Proposition: "The researchers conducted analysis."
Evaluation: accuracy: 7, clarity: 6, completeness: 4, conciseness: 9

Format:
Proposition: "{proposition}"
Original Text: "{original_text}"
```

Claim checking – examples of prompts

```
You are a scientific literature search expert. Given this scientific claim:
↪  "{claim}"

Generate two related claims:

1. OPPOSITE: A statement that contradicts the original claim.
   Use natural phrasing (e.g., "has no effect", "is not linked to", "may
   ↪  worsen", "fails to", "increases").
   Keep it concise and scientifically plausible.

2. NEUTRAL: A statement that is related but does not agree or disagree.
   Include core and related keywords that could appear in scientific papers
   ↪  on the same topic
   (e.g., synonyms, broader or adjacent concepts).

Respond in this exact format:
OPPOSITE: <text>
NEUTRAL: <text>

Example:
Claim: "Green tea improves memory."
OPPOSITE: Green tea shows no significant effect on memory function.
NEUTRAL: Studies explore green tea, cognition, neuroprotection, brain health
↪  and memory-related biomarkers.

Now generate for the given claim:
```

Literature search: diverse query generation

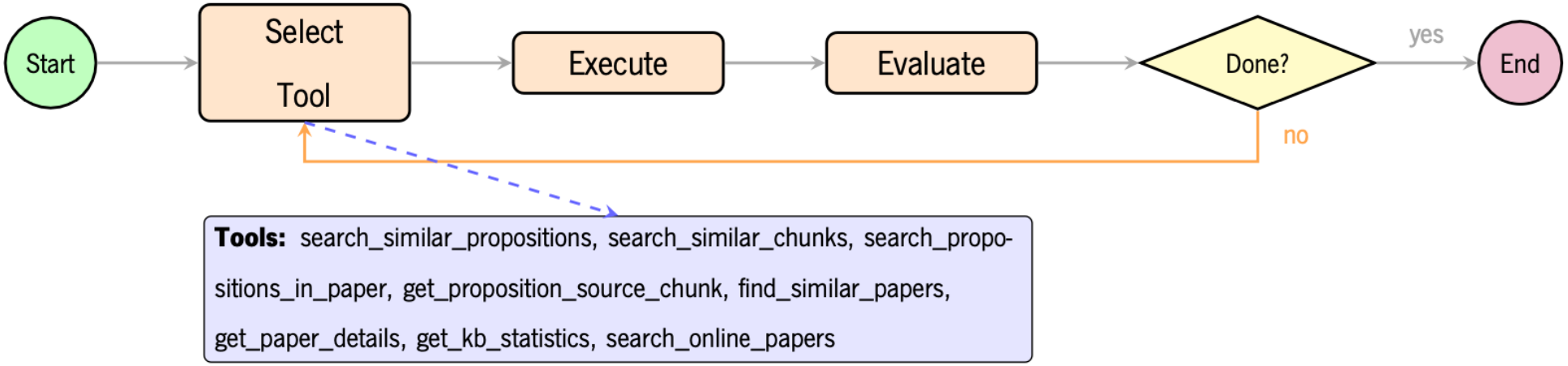
Claim checking – examples of prompts

```
You are a scientific claim verification system. Your task is to evaluate
↪  whether a given claim is supported, refuted, or has insufficient evidence
↪  based on the provided scientific evidence.

VERIFICATION GUIDELINES:
- **SUPPORTS**: The evidence clearly supports the claim with specific facts,
↪  data, or findings
- **REFUTES**: The evidence directly contradicts the claim with contrary
↪  facts or data
- **INSUFFICIENT_EVIDENCE**: The evidence is irrelevant, too vague,
↪  conflicting, or does not provide enough reliable information to verify
↪  the claim

When determining your verdict, always base your reasoning on the relationship
↪  between the claim and the retrieved evidence, not on external world
↪  knowledge.

CONFIDENCE SCORING (1-10)
Rate your confidence in the verdict according to how clearly and consistently
↪  the retrieved evidence justifies it

For SUPPORTS verdict:
- 9-10: Extremely confident the claim is true - overwhelming, consistent
↪  evidence from high-quality sources
```

....

Claim verification

Claim checking – autonomous agent




```
You are a scientific claim verification agent. Your task is to verify whether
↪  a scientific claim is supported, refuted, or has insufficient evidence.

AVAILABLE TOOLS:
{tools_available}

YOUR APPROACH:
You have autonomy in how you investigate claims. Adapt your strategy based on
↪  what you find. However, keep these principles in mind:

- Start by understanding what's available (get_kb_statistics gives you an
↪  overview)
- Search broadly first, then dive deeper into promising papers
- Actively look for BOTH supporting AND contradicting evidence - don't stop
↪  at the first match
```

...

Some useful libraries for LLMs

- **AI suite** (https://github.com/andrewyng/aisuite) - use multiple LLM through a standardized interface
- **Langchain** (https://github.com/langchain-ai; https://github.com/langchain-ai/langchain; https://github.com/langchain-ai/langgraph) - framework for building LLM-powered applications (RAG, agents, etc.)
- **PromptBench** (https://github.com/microsoft/promptbench) – evaluation of LLMs (pytorch)
- **GraphRAG** (https://github.com/microsoft/graphrag) – extraction of information from texts
- **Argilla** (https://github.com/argilla-io/argilla) – building datasets to evaluate
- **HuggingFace** (https://huggingface.co/docs/hub/index) – model repository and many other resources
- **AutoGen** (https://microsoft.github.io/autogen/) - enables the development of LLM applications using multiple agents

More complete list in: https://www.promptingguide.ai/research/llm-agents